NEST: Network Enforced Session Types

Jens Kanstrup Larsen
Technical University of Denmark, Copenhagen, Denmark

Alceste Scalas
Technical University of Denmark, Copenhagen, Denmark

Guy Amir
Cornell University, Ithaca, NY, USA

Jules Jacobs
Cornell University, Ithaca, NY, USA

Jana Wagemaker
Radboud University, Nijmegen, The Netherlands

Nate Foster
EPFL, Lausanne, Switzerland

Abstract

This paper introduces NEST (Network Enforced Session Types), a runtime verification framework that moves application-level protocol monitoring into the network fabric. Unlike prior work that instruments or wraps application code, we synthesise packet-level monitors that enforce protocols directly in the data plane. We develop algorithms to generate network-level monitors from session types and extend them to handle packet loss and reordering. We implement NEST in P4 and evaluate it on applications including microservice and network-function models, showing that network-level monitors can enforce realistic non-trivial protocols.

2012 ACM Subject Classification Software and its engineering → Software verification

Keywords and phrases Session types, runtime verification, P4, programmable data planes

Digital Object Identifier 10.4230/LIPIcs.ECOOP.2026.17

Related Version *Technical Report*: https://doi.org/10.48550/arXiv.2604.21795

Supplementary Material *Software (ECOOP 2026 Artifact Evaluation approved artifact)*: https://doi.org/10.4230/DARTS.12.1.2

Funding Research partially supported by: the DTU Nordic Five Tech Alliance grant "Safe and secure software-defined networks in P4"; the Horizon Europe grant no. 101093006 "TaRDIS"; the Independent Research Fund Denmark project "Hyben"; the DARPA grant no. W912CG-23-C-0032 "LANCER"; and the Dutch research council (NWO) under grant no. VI.Veni.242.134 (VerHyp). The work of Amir was partially supported by a Rothschild Fellowship from Yad Hanadiv (The Rothschild Foundation).

1 Introduction

Session types are a well-established formalism for specifying and verifying message-passing programs [26, 27, 19, 68, 10]. Whereas conventional type systems model the types of data used by each process (i.e., integers, strings, objects, etc.), session types also model how processes interact by sending and receiving messages. E.g., a session type may specify that a process should receive a string from A, send an integer to B, then receive a boolean from C.

© Jens Kanstrup Larsen, Alceste Scalas, Guy Amir, Jules Jacobs, Jana Wagemaker, and Nate Foster;
licensed under Creative Commons License CC-BY 4.0

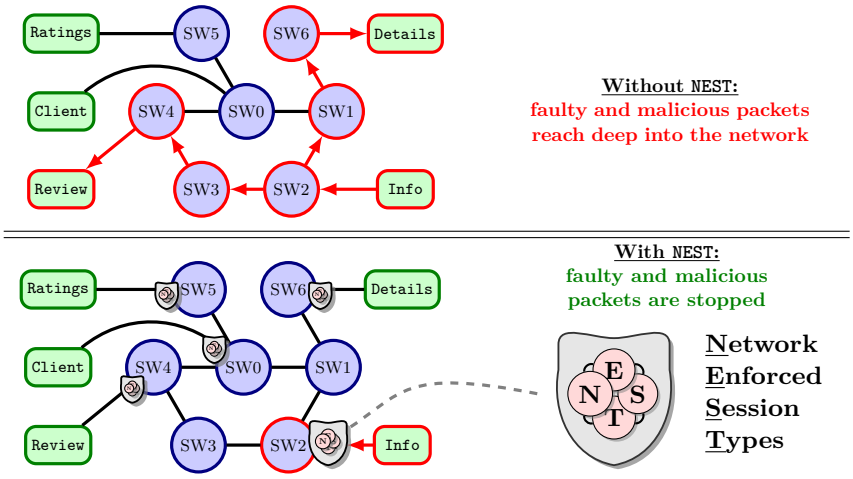


Figure 1 A network of microservices based on the Istio "BookInfo" application [34], highlighting a flow of bad packets generated by a faulty implementation of the `Info` microservice, and how NEST can stop them.

Session types have been used to verify implementations of complex multiparty protocols, ensuring that each node only sends and receives well-typed messages and that the system does not fail unexpectedly. Although they originated in process algebras, session types have been incorporated into mainstream programming languages including Rust, Go, Java, OCaml, and Scala [12, 55, 11, 31, 37, 41, 15, 16, 28, 29, 62, 57, 56, 59, 32, 69].

Network-level session monitoring: opportunities and challenges. A *monitor* observes a system at runtime and checks conformance to a specification. For a session type T, a monitor observes sent and received messages; on a violation, it can raise an alert or drop the message. Monitors are useful if programs running on certain nodes cannot be statically verified, or for providing defense in depth. Most prior work has focused on monitoring at the application level [8, 9, 6, 53, 17, 54]. Here we ask a different question: *can we synthesise monitors that enforce session types at the network level?* We have two primary motivations.

First, deploying monitors deeper in the network stack places them beyond end-host control. This gives stronger assurance in mixed-trust settings: in a public cloud, provider-managed monitors can enforce session types even when tenants do not trust one another. This motivation is illustrated in Fig. 1, which shows a network based on the "BookInfo" application described by Istio [34]. The network includes four end hosts, each implementing a different microservice (`Info`, `Review`, `Details`, and `Ratings`) connected to each other and to an external `Client`. The `Client` queries the `Info` end host for information on a book; to answer the query, `Info` in turn queries `Review` and `Details`, and `Review` also queries `Ratings`. Without network-level session monitoring, faulty or malicious code running on one of the end hosts (e.g., `Info`) may generate invalid packets that reach other end hosts, consuming network resources and potentially crashing applications when they receive unexpected messages.

Second, network-level monitors can run efficiently on suitable hardware, such as programmable switches and NICs. For instance, the P4 language for programming network switches [7] is designed for high performance, line-rate processing of packets [30, 38].

Notably, since session types describe application-level protocols, their monitoring at the network level breaks the classical network layering. This is not uncommon in modern networks, where classical layering abstractions are sometimes broken to implement various

functionalities. Middleboxes such as NAT boxes, load balancers, proxies, and content caches realise functionality at the transport layer (i.e., TCP/UDP or Layer 4) or above, by manipulating packets at the network layer (i.e., IP or Layer 3) [65]. Moreover, existing network-layer devices already enforce simple application-level patterns – e.g., NAT boxes forward packets from external hosts only in response to communication initiated by internal hosts. Compared to these approaches, session types can express richer protocols and come equipped with formal guarantees.

Realising network-level session type monitors requires addressing several challenges:

- Session types can model rich behaviours that go well beyond static policies and simple firewalls. Hence, monitor synthesis must be *automatic*.
- Monitors for session types must also be *stateful*, to track the current protocol state and accept or reject packets accordingly. While basic connection tracking exists in devices such as stateful firewalls and NAT boxes, session types bring additional complexity.
- Network-level monitors must handle reordered packets and retransmissions after *loss*, often with limited hardware buffering. By contrast, existing application-level session monitors assume reliable transport (e.g., TCP) and enough buffering to reorder packets.

Contributions and outline. To address these challenges, we design and implement `NEST`, a tool for generating and deploying network-level session monitors. `NEST` takes as input a set of local session types and their associated roles, generates corresponding P4 monitor representations, and deploys them on P4-enabled network devices. We develop the formal foundations of `NEST`, and evaluate it on a set of representative multiparty protocols using Mininet [44], a realistic software-defined networking emulation platform. By enforcing session types at the network perimeter, `NEST` monitors can discard protocol-violating packets early, saving network resources and preventing invalid traffic from reaching downstream hosts and devices. More broadly, our approach realises an "off-by-default" network [1], where only authorised packets can traverse the network. The main contributions of this paper are:

§3 introduces `NEST`. Given a set of session types written in a Scala 3 embedded DSL, `NEST` synthesises network-layer monitors based on P4. `NEST` also generates an API for writing end-host programs whose communication patterns can be tracked by our monitors (§3.4).

§4 presents the formal model at the basis of `NEST`. We introduce a novel monitor synthesis technique (Def. 4.10) for network-level session monitors that reject bad packets while ensuring *soundness*: messages from well-behaved end hosts are not rejected (Theorem 4.18).

§5 presents a detailed evaluation of `NEST`. Using a variety of representative multiparty protocols, we show that `NEST` monitors accept valid packets and reject invalid ones.

Finally, §6 discusses related work, and §7 concludes with future directions. `NEST` is available in the companion artifact of this work with instructions for reproducing the evaluation in §5. Additional examples, evaluation results, and proofs are available in a technical report [46].

2 Background

We briefly review session types using the Istio BookInfo application [34] from §1, a microservice application for an online bookseller. First, a `Client` requests information on a given book. The outward-facing `Info` service then gathers data from two internal microservices: `Details` and `Review`. The `Details` microservice replies immediately with data (e.g., author and ISBN). Meanwhile, `Review` first queries the internal `Ratings` service and then replies with

```
protocol ReviewProtocol {
  role Client, Info, Review, Ratings, Details;
  Client -> Info: request(int);
  Info -> Review: review_request(int);
  Info -> Details: detail_request(int);
  Review -> Ratings: ratings_request(int);
  Ratings -> Review: ratings_response(float);
  Review -> Info: review_response(str);
  Details -> Info: detail_response(str);
  Info -> Client: response(str);
}
```

Figure 2 BookInfo protocol as a global type.

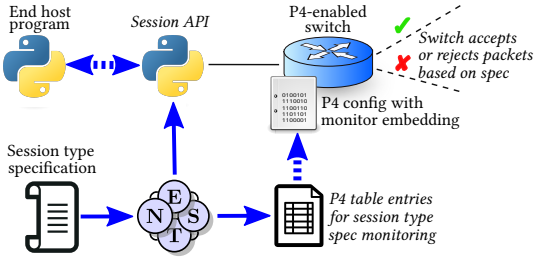


Figure 3 Overview of NEST: the programmer writes a (local) session type specification (left); NEST generates P4 switch configurations, which drop non-conformant packets (right). NEST also generates a Python session API, which end-host programs can use for network communication.

```
ReviewProtocol[Info] = {
    Client?request(int);
    Review!review_request(int);
    Details!detail_request(int);
    Review?review_response(str);
    Details?detail_response(str);
    Client!response(str);
}
```

```
sh = BookReviewSessionHandler(42, Info)
while True:
  book_id = sh.recvMsg(Client, request)
  sh.sendMsg(Review, review_request(book_id))
  sh.sendMsg(Details, detail_request(book_id))
  review = sh.recvMsg(Review, review_response)
  detail = sh.recvMsg(Details, detail_response)
  sh.sendMsg(Client, response(review + detail))
```

Figure 4 Local session type (left) and end host code (right) for the `Info` role in the BookInfo application [34]. The `?` and `!` symbols represent receiving and sending a message, respectively.

reviews and ratings. The behaviour of BookInfo can be described with the global type in Fig. 2: `Client` sends `Info` a `request` message with `int` payload; `Info` then sends `Review` a `review_request(int)` message, and so on.[1]

By *projecting* a global type into one role, we obtain a *local session type*. The local session type in Fig. 4 describes the communication protocol enacted by role `Info`. Beyond this simple example, session types can also express *branching* and *recursion* (formalised in Def. 4.1 later on): branching introduces choice points where a role may send or receive one among several different messages and possibly continue the session in a different way (see Example 4.7), while recursion allows for repeating part of a session (see Example 4.13). Local session types are typically used for compile-time type checking. In practice, many components may not be session-typed (e.g., unsupported languages/frameworks or inaccessible participants such as the external `Client` in BookInfo). Still, local session types remain a precise and expressive protocol specification language, so we use them as the basis for network monitoring.

3 NEST: Overview and End-Host Monitor Design

This section overviews NEST, our toolkit for synthesising and deploying network-level monitors from session type specifications. Fig. 3 summarises the workflow.

[1] The syntax of the global type in Fig. 2 is inspired by tools like Scribble (`https://github.com/scribble/scribble-java`) and νScr (`https://github.com/nuScr/nuScr`) and only serves to illustrate the standard session types framework and the BookInfo protocol. As explained later in §3 and Remark 4.19, NEST takes as input one or more *local* session types; if needed, such local session types can be *projected* from a global type using the standard techniques mentioned in this section, using tools like Scribble, νScr, and mpstk (`https://github.com/alcestes/mpstk-crash-stop`).

Our design for NEST relies on two key assumptions. First, we assume that the intended behaviour of each end host is captured by a local session type T; for now, the intuitive understanding of session types from §2 will suffice (the formal definition will be given in §4). Second, we assume that the behaviour of the devices at the edge of the network can be specified in P4, a domain-specific language for programming network switches [7]. To understand NEST, a deep understanding of P4 will not be necessary. For now, there are two things to know: (i) P4 is based on *tables* which can be populated with entries at runtime to control how packets are processed; (ii) P4 provides mutable registers, with associated read and write operations, which can be used to implement stateful packet processing. We will use both of these features in our design for NEST.

Given an application's (local) session type, NEST generates:

1. A set of **P4 routing table entries** that monitor and enforce the session type on incoming traffic, while tracking the session state progression as packets are sent/received by participants.
2. An **API** for sending and receiving messages in the format expected by our network monitors. We describe the message format in §3.1 and the session API generation in §3.4.

NEST monitors reject illegal packets at the network perimeter. For example, in Fig. 1, traffic from the faulty node is dropped at switch SW2.

Given a session type such as Fig. 5 and a role, NEST synthesises a monitor in four steps that "bring down" an application-level session type specification into the network layer:

1. NEST's synthesis module constructs a state machine for the session-type monitor.
2. NEST converts this state machine into the entries of a Match-Action Table (MAT). Intuitively, for each row of the MAT, the "match" column specifies how to match a packet based on (1) the current state of the switch and (2) the label, sender and receiver carried by the packet. Then, the "action" column specifies whether to accept the packet (transitioning to another state) or reject it. Fig. 6 shows the MAT generated for Fig. 5.
3. NEST then translates these MAT entries into entries for the P4 table `monitor_table` (§3.1), encoding sender/receiver roles and message labels as enumerated IDs. NEST also provides additional P4 monitoring logic for handling packet loss, duplication, and reordering in TCP connections (§3.3).
4. Finally, NEST deploys the generated entries on P4-enabled devices acting as monitors.[2]

After deployment, the P4 device enforces session-type monitoring: upon receiving a packet, `monitor_table` inspects the packet header and accepts or rejects it. NEST also generates a session API for sending/receiving packets with the headers expected by the monitors (§3.4).

Challenges. The rest of this section addresses three practical challenges.

- **Accept/reject decisions and session state tracking (§3.1):** how should a P4 device correctly accept/reject packets while tracking multiple session types concurrently?
- **Shared entry points (§3.2):** how should one monitor handle multiple end hosts sharing an ingress point?
- **Packet loss, duplication, and reordering (§3.3):** how should NEST support transport protocols (e.g., TCP) that affect packet sequencing?

[2] The deployment phase uses the P4R-Type library [45] to statically ensure that the deployed entries conform to the `monitor_table` specification in P4.

```
val lst_BookInfo =
  ExtCh(a_BookClient, Map(l_Request ->
  IntCh(Map((a_BookReview, l_ReviewRequest) ->
  IntCh(Map((a_BookDetails, l_DetailRequest) ->
  ExtCh(a_BookReview, Map(l_ReviewResponse ->
  ExtCh(a_BookDetails, Map(l_DetailResponse ->
  IntCh(Map((a_BookClient, l_Response) ->
  End() ))))))))))))
```

Figure 5 The `Info` session type in Fig. 4 (left), represented as a Scala 3 data type instance which `NEST` takes as input. `ExtCh` means "await an incoming message chosen and sent by another role," whereas `IntCh` means "choose a recipient role and a message, and perform the send operation." (In this example, all choices have only one option.).

Match				Action
State	Sender	Receiver	Label	
m0	Client	Info	req	accept(m1)
m1	Info	Review	r_req	accept(m2)
m2	Review	Info	r_rsp	accept(m3)
m2	Info	Details	d_req	accept(m4)
m3	Info	Details	d_req	accept(m5)
m4	Review	Info	r_rsp	accept(m5)
m4	Details	Info	d_rsp	accept(m6)
m5	Details	Info	d_rsp	accept(m7)
m6	Review	Info	r_rsp	accept(m7)
m7	Info	Client	rsp	accept(m8)
Otherwise				reject

Figure 6 A match-action table (MAT) generated by `NEST` from the session type in Fig. 5. The message labels are shortened for brevity. In the "action" column, accept(mX) indicates that the monitor transitions to state mX upon accepting the message.

Assumptions and limitations. We assume the devices at the edge of the network can be programmed in P4, so that all communication between protocol roles passes through monitored devices. The current version of `NEST` also assumes each session message fits into a single packet; ideas for lifting this restriction to handle fragmentation and additional transport protocols are discussed in §7.

3.1 Accepting/Rejecting Packets and Tracking Session-Type State

`NEST`-generated monitors decide whether to accept or reject a packet by processing a dedicated *session header* (Fig. 7). Each monitored-session packet must carry this header; others are rejected by default. The P4 monitoring logic is implemented by the table `monitor_table` (Fig. 8), which extracts from the session header the *sender role*, *message label ID*, *receiver role* – and obtains the *current state* from the stateful register on the switch – to decide whether to accept or reject the packet..[3] The `monitor_table` matches extracted session-header fields against entries generated by `NEST` from a session-type MAT such as Fig. 6. Each packet can match at most one `monitor_table` entry – and if so, the packet is accepted; otherwise, the default `reject` action drops the packet (line 13 in Fig. 8).

The `accept` action of `monitor_table` also inspects the *session ID*, allowing one device to distinguish and track many sessions in parallel: if a packet with session ID s is accepted, `accept` uses s to index a P4 register and update that session's state. The session header also carries a *session sequence number*, used to handle retransmissions (discussed in §3.3).

Computing the session monitor MAT. `NEST`'s monitoring strategy hinges on correctly synthesising the session-type monitor MAT that determines which packets are accepted or rejected. For instance, the state machine in Fig. 9a corresponds to the standard semantics of the session type in Fig. 5 – whereas the state machine in Fig. 9b is synthesised by `NEST` from the same session type, and then converted into the MAT in Fig. 6. Observe that state

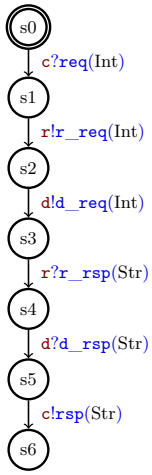
[3] The session header currently used by `NEST` (depicted in Fig. 7) supports protocols with up to 15 distinct roles and up to 63 distinct message labels. This bound can be increased if needed, although doing so may be constrained by packet size and the memory available on the P4 switch.

0	1	2	3	4	5	6	7
Message Size							
Session ID							
		Message Label ID					
Sender Role				Receiver Role			
Session Sequence Number							

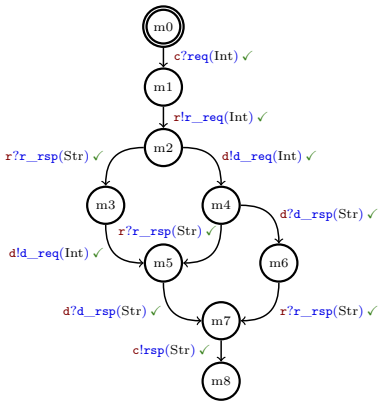
Figure 7 Specification of the session header.

```
table monitor_table {
  key = {
        hdr.session.sender: exact;
        hdr.session.label: exact;
        hdr.session.receiver: exact;
        currentState: exact;
    }
    actions = {
        accept;
        reject;
    }
    size = 1024;
    default_action = reject();
}
```

Figure 8 Specification of the P4 table for monitor transitions.



(a) The state machine of the session type in Figures 4 and 5 (following the formal semantics of $T_{\texttt{Info}}$ by Def. 4.3).



(b) The state machine of the monitor for the session type in Figures 4 and 5 (following the formal semantics of $[\![T_{\texttt{Info}}]\!]$ by Def. 4.10). Note that this diagram only shows the monitor's accepting transitions; it omits the transitions that reject any send/receive action which is not explicitly accepted (by rule STMon-Bad in Def. 4.10).

Figure 9 The state machines of the session type in Figures 4 and 5 and its monitor. For brevity, we shorten the message labels and write `c`, `r`, and `d` for `Client`, `Review`, and `Details`, respectively.

m2 of the MAT allows `Info` to either send `d_req` to `Details` or receive `r_rsp` from `Review` – whereas these actions are sequential in Figures 5 and 9a (first send, then receive). The difference between Fig. 9a and Fig. 9b is a consequence of "bringing down" a session type specification to the network layer for monitoring purposes. This is because, while enforcing a session type T, `NEST`'s network-level monitor may see packets that diverge from T's expected order. The monitor must distinguish "bad" packets that violate T from "good" packets delivered in a different order. We formalise this in §4 and prove correctness.

3.2 Shared Entry Points

Although Fig. 1 shows one switch per end host, `NEST` also supports multiple end hosts per switch, even with different roles. This raises a practical challenge: *how can one device monitor multiple roles concurrently?* Suppose a device must monitor roles `p` and `q` in session types $T_{\texttt{p}}$ and $T_{\texttt{q}}$. Let $[\![T_{\texttt{p}}]\!]$ and $[\![T_{\texttt{q}}]\!]$ be the monitors synthesised when these roles are monitored separately. We build one monitor from these two using standard process-calculus techniques:

1. Compute the labelled transition system (LTS) of the *parallel composition* $[\![T_{\mathtt{p}}]\!] \parallel [\![T_{\mathtt{q}}]\!]$ in the style of CCS [50], allowing synchronisation when the monitor $[\![T_{\mathtt{p}}]\!]$ accepts an outgoing message to $[\![T_{\mathtt{q}}]\!]$ (or *vice versa*).
2. Prune non-synchronising transitions where the monitor $[\![T_{\mathtt{p}}]\!]$ accepts a message to/from $[\![T_{\mathtt{q}}]\!]$ (or *vice versa*); in CCS terms, apply restriction $(\nu\, \mathtt{p},\mathtt{q})\,([\![T_{\mathtt{p}}]\!] \parallel [\![T_{\mathtt{q}}]\!])$.
3. Deploy on the P4 device a monitor state machine matching this restricted composition, so the device can monitor $T_{\mathtt{p}}$ and $T_{\mathtt{q}}$ together.[4]

3.3 Handling Packet Loss, Duplication, and Reordering

So far, NEST-generated monitor state machines and MATs (Figures 6 and 9b) cover the "core logic" from §3.1: for a session type T, they distinguish violating packets from out-of-order but valid packets. This is also the focus of the formal synthesis in §4. The result is a P4 monitor that works on reliable networks.

However, such monitors would often behave incorrectly in real-world networks, where packets may be lost, duplicated, and reordered (even when they have the same sender). To address these issues, transport protocols such as TCP add transmission logic to packets and may adjust their sending order to guarantee reliable delivery. Therefore, NEST is designed to be flexible with respect to transport protocols between end hosts. In particular, NEST can generate *TCP-oriented P4 monitors for session types* by augmenting a "core" monitor state machine and MAT (like those in Figures 6 and 9b, which follow Def. 4.10) with transitions and checks for TCP socket setup/teardown, packet acknowledgements, and retransmissions within a TCP connection. E.g., Fig. 10 depicts the TCP-oriented monitor obtained from the state machine in Fig. 9b by adding TCP-specific transitions (dashed arrows) and checks.

- The TCP-oriented monitor always accepts packets related to TCP connection setup and teardown (i.e., TCP headers with the `SYN` or `FIN` flag set) without changing its state. It also accepts pure TCP acknowledgements (i.e., TCP headers with `ACK`) unless the `ACK` is piggybacked with a session header, in which case normal monitoring applies.
- As mentioned in §3.1, the session header carries a *session sequence number (SSN)* field, which records the total number of messages sent by the sender so far. These sequence numbers are tracked by our TCP-oriented monitors. The TCP-oriented monitor uses a detailed decision procedure based on SSNs; the details of this decision procedure are simplified away for the transitions shown in Fig. 10, but are explained in Fig. 11 (essentially, the end states in Fig. 11 correspond to transitions that each state in Fig. 10 can take). We explain the decision procedure:
  1. If an incoming packet's SSN is less than or equal to the monitor's stored SSN for that sender, the packet is accepted as a retransmission and does not update monitor state.
  2. Otherwise (SSN greater), the packet is first matched on sender and recipient *only*.[5]

[4] With a minor extension of the formal model introduced later in §4, the behaviour of a network using this joint monitor for $T_{\mathtt{p}}$ and $T_{\mathtt{q}}$ can be proven bisimilar to a network where the end hosts of `p` and `q` are connected to different devices, matching the premises of our monitor correctness result (Theorem 4.18).

[5] Step 2 ignores the packet label because if the monitor were to match simultaneously on sender, recipient, and label at this step, then a failed match would be ambiguous and would not allow the monitor to distinguish between two cases: truly invalid packets that must cause the session to be dropped (handled in step 4), or possibly out-of-order packets that must be dropped to be later retransmitted (handled in step 3). To match packets only on sender and recipient, the TCP-oriented monitor uses an additional table, `receiver_table`, a partial version of `monitor_table` that ignores the packet label.

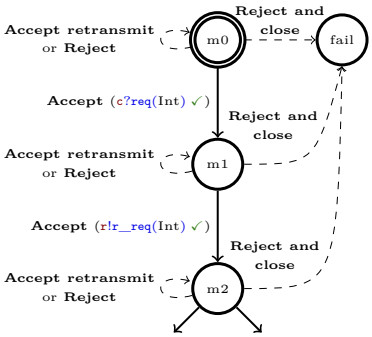


Figure 10 Initial states and transitions of the TCP-oriented monitor generated from Fig. 9b.

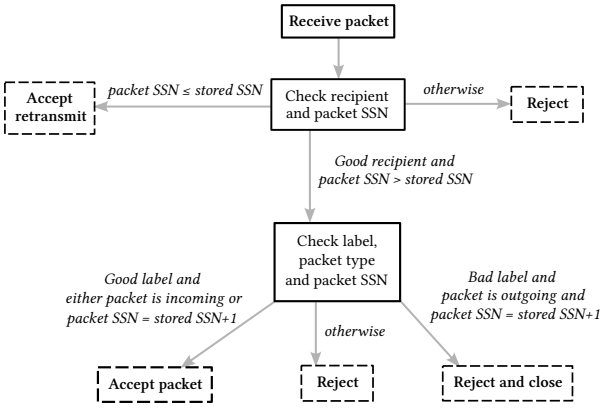


Figure 11 A flowchart showing how the TCP-oriented monitor processes incoming packets.

3. If both sender and recipient match, the monitor then matches the packet label. If that matches, there are two cases. Suppose that the monitor is guarding the network border for end host p:
   - For each outgoing packet sent by p, if the packet SSN matches the next expected value (i.e., the stored SSN plus one), the monitor accepts the packet and updates its state. On the other hand, if the SSN is too high, the monitor drops the packet: this is because the SSN being too high means that some packets were lost, hence the monitor awaits their retransmission with the correct SSN.
   - For each incoming packet towards p, the monitor trusts the SSN as-is and updates its state without further checks: the monitor assumes that the SSN was already checked and accepted by the border monitor on the sender side.
4. If both sender and recipient match but the label does not, the packet is rejected. Moreover, if the monitor is guarding end host p and the packet is sent by p with an SSN that is exactly one higher than the stored SSN, the monitor rejects the packet *and* permanently closes the session – because this indicates a session violation (not mere packet retransmission or reordering).[6]

With this approach, our TCP-enabled monitors can handle packet loss (since TCP eventually retransmits lost and unacknowledged packets), duplication (treated as a special case of retransmission), and reordering of packets from the same sender (by simply ignoring out-of-order packets and waiting for their retransmission).

Limitations. Our approach to monitoring TCP connections has limitations:

- Our TCP-oriented monitors let all `ACK` packets pass through. A malicious end host might abuse this to flood the network with spoofed ACK messages, which the monitors would not reject. This is a common risk in networks with TCP services, and it may require mitigations such as rate limiting against TCP-based DoS (denial-of-service) attacks.
- Since the TCP-oriented monitors do not reject packets with low session sequence numbers, a malicious or faulty sender could send messages with low session sequence numbers through the network, as they would not be rejected at the border.[7] To mitigate this,

[6] Closing the session ensures the bad message is not later accepted as a retransmission after SSN advances.
[7] Packets with lower-than-expected SSN are ignored by programs that use `NEST`-generated APIs (§3.4).

monitors could rate-limit such packets, since they can be treated as retransmissions. The rate-limiting design is orthogonal to our monitoring logic, so we leave it to future work.

- NEST does not currently support packet fragmentation; we discuss remedies in §7.

3.4 API Generation for Session-Monitored Applications

Because our monitors require packets to carry the session header described in Sections 3.1 and 3.3, NEST generates an API for writing monitor-compliant applications. The API hides session-header details and exposes human-readable message-label constants derived from the input session type, rather than numeric label IDs. Our current prototype targets Python and supports the send/receive style in Fig. 4. By design, the API generated by NEST is minimalistic and does not enforce the ordering of send/receive operations specified in the input session type. We take advantage of this in our evaluation (§5.2) to write programs that do *not* follow a session type and show that NEST monitors correctly reject their messages.

The generated API is based on a `SessionManager` class that instantiates `Session` objects, each representing one session. Each `Session` is created with a protocol and a session ID. The API also maintains a queue of incoming messages, from which `Session` objects dequeue via `recvMsg()`. This prevents the host program from incorrectly dropping messages (e.g., when a message is delivered earlier than expected).

Automatic Session ID Propagation. The `SessionManager` also handles the automatic *propagation* of session IDs. If a program creates a `Session` without an ID, that session adopts the ID from the first incoming message carrying an ID not in use on that `SessionManager`. Subsequent messages sent over that `Session` propagate that ID, and the `Session` dequeues only messages with that ID. To support ID propagation, the session protocol must have an *initiator* role that sends the first message(s) to one or more peers, who learn and propagate the ID carried by such messages. All protocols in our evaluation (§5) follow this pattern.

4 Proving the Correctness of NEST Monitors

In this section, we establish the correctness of NEST's monitor synthesis, i.e., the monitor state machine and MAT outlined at the end of §3.1. We focus on two key challenges for defining the "core logic" of our network-layer monitors:

1. Monitors must correctly accept or reject packets *immediately*, without resorting to buffering – which is assumed in previous work on session monitoring [6, 17] but is often infeasible in network devices due to memory limitations.
2. Monitors must make correct decisions even when packets from different senders are delivered in an order that does not match the expectations of the end hosts.

To isolate these challenges, we study monitor synthesis and correctness in an idealised network where messages are delivered instantaneously, without fragmentation, loss, duplication, or same-sender reordering; as explained in §3.3, NEST handles these aspects[8] by augmenting the "core logic" of the monitors with additional checks and transitions tailored to TCP as a transport protocol. Challenges 1 and 2, instead, fundamentally affect monitoring logic, independently of the transport protocol in use.

[8] Except packet fragmentation, which is less common in modern networks configured with a consistent maximum transmission unit (MTU). Support for fragmentation is future work discussed in §7.

In §4.1 we formalise networks where end-host behaviours are modelled as session types and protocol-violating packets may still reach hosts. In §4.2, monitors block those packets. §4.3 then formalises our monitor synthesis (Def. 4.10). Finally, §4.4 proves soundness: synthesised monitors do not reject traffic when all end hosts follow the enforced protocol (Theorem 4.18).

4.1 Session Types and Networks

In Definitions 4.1 and 4.3 we model a network end host as a (local) session type with a *multi-input queue* that stores incoming messages from multiple senders, while preserving the order of messages from each sender [18]. The idea is that the session type models the behaviour of a message-passing program, while the queue models the end host's ability to buffer incoming messages (e.g. in its network stack). This modelling is standard in the session-types literature, except that multi-output queues are often used instead of input queues.

▶ **Definition 4.1** (End host model). *The syntax of session types with multi-input queues is:*

$$
\begin{array}{rrcl}
\textit{Payload type (a.k.a. sort)} & S & ::= & \mathrm{Bool} \mid \mathrm{Int} \mid \mathrm{Str} \mid \cdots \\
\textit{Role} & \mathtt{p}, \mathtt{q}, \mathtt{r} & ::= & \mathtt{Info} \mid \mathtt{Client} \mid \mathtt{Review} \mid \mathtt{Details} \mid \cdots \\
\textit{Session type} & T & ::= & \sum_{i \in I} \mathtt{p}_i \oplus \mathtt{m}_i(S_i).T_i \\
 & & \mid & \mathtt{p} \,\&\, \&_{i \in I} \mathtt{m}_i(S_i).T_i \\
 & & \mid & \mathbf{end} \\
 & & \mid & \mu \mathbf{t}.T \\
 & & \mid & \mathbf{t} \\
\textit{Input queue type} & \sigma & ::= & \mathtt{p} \triangleright \mathtt{m}(S) \cdot \sigma \mid \epsilon \\
\textit{Session type with input queue} & \ddot{T} & ::= & T \langle\!| \sigma
\end{array}
$$

where $I \neq \emptyset$ and the message labels $\mathtt{m}_i$ *are pairwise distinct. We require session types to be closed and recursion variables to be guarded.* ⌟

The type $\sum_{i \in I} \mathtt{p}_i \oplus \mathtt{m}_i(S_i).T_i$ is an *internal choice* where the end host selects one recipient role $\mathtt{p}_i$ and sends a message with label $\mathtt{m}_i$ carrying payload type S_i; then, the interaction continues as specified in T_i. Dually, $\mathtt{p} \,\&\, \&_{i \in I} \mathtt{m}_i(S_i).T_i$ is an *external choice* where the end host awaits a message $\mathtt{m}_i$ with payload type S_i from sender $\mathtt{p}$; then, the interaction continues as T_i. The lists of possible roles $\mathtt{p}, \mathtt{q}, \mathtt{r}$ and payload types S are examples that can be extended as needed. The type **end** represents a terminated session, while $\mu \mathbf{t}.T$ and $\mathbf{t}$ represent recursion. We define $T \langle\!| \sigma$ as the pairing of a session type with a multi-input queue σ, where each element $\mathtt{p} \triangleright \mathtt{m}(S)$ represents a message $\mathtt{m}$ sent by role $\mathtt{p}$ with payload type S.

▶ **Example 4.2.** The formal definition of the session type $T_{\mathtt{Info}}$ for the role `Info` from Fig. 4 is the following (for brevity, we shorten role names and message labels):

$$T_{\mathtt{Info}} \;=\; \mathtt{c} \,\&\, \mathtt{req}(\mathrm{Int}).\ \mathtt{r} \oplus \mathtt{r_req}(\mathrm{Int}).\ \mathtt{d} \oplus \mathtt{d_req}(\mathrm{Int}).\ \mathtt{r} \,\&\, \mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d} \,\&\, \mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c} \oplus \mathtt{rsp}(\mathrm{Str})$$

⌟

▶ **Definition 4.3** (Semantics of session types with multi-input queues). *The labelled transition system (LTS) semantics of session types (without queues) is defined as follows, using the labels $\alpha ::= \mathtt{p} \oplus \mathtt{m}(S) \mid \mathtt{p} \& \mathtt{m}(S)$:*

$$\frac{k \in I}{\sum_{i\in I} \mathtt{p}_i \oplus \mathtt{m}_i(S_i).T_i \xrightarrow{\mathtt{p}_k \oplus \mathtt{m}_k(S_k)} T_k}\ \textit{S-IntC} \qquad \frac{k \in I}{\mathtt{p}\&_{i\in I}\mathtt{m}_i(S_i).T_i \xrightarrow{\mathtt{p}\&\mathtt{m}_k(S_k)} T_k}\ \textit{S-ExtC}$$

$$\frac{T\{\mathbf{t} \mapsto \mu\mathbf{t}.T\} \xrightarrow{\alpha} T'}{\mu\mathbf{t}.T \xrightarrow{\alpha} T'}\ \textit{S-Rec}$$

The LTS semantics of session types with input queues is defined as follows:

$$\frac{T \xrightarrow{\mathtt{p}\oplus\mathtt{m}(S)} T'}{T \langle\!| \sigma \xrightarrow{\mathtt{p}!\mathtt{m}(S)} T' \langle\!| \sigma}\ \textit{SQ-Send} \qquad \frac{T \xrightarrow{\mathtt{p}\&\mathtt{m}(S)} T' \quad \sigma \equiv \mathtt{p}\triangleright\mathtt{m}(S)\cdot\sigma'}{T \langle\!| \sigma \xrightarrow{\tau(\mathtt{p}\&\mathtt{m}(S))} T' \langle\!| \sigma'}\ \textit{SQ-Deq}$$

$$\frac{}{T \langle\!| \sigma \xrightarrow{\mathtt{p}?\mathtt{m}(S)} T \langle\!| \sigma\cdot\mathtt{p}\triangleright\mathtt{m}(S)}\ \textit{SQ-Recv}$$

where τ denotes an internal transition that does not synchronise with others; in SQ-Deq, $\equiv$ is the smallest congruence s.t. $\mathtt{p} \neq \mathtt{q}$ *implies* $\mathtt{p}\triangleright\mathtt{m}(S)\cdot\mathtt{q}\triangleright\mathtt{m}'(S')\cdot\sigma \equiv \mathtt{q}\triangleright\mathtt{m}'(S')\cdot\mathtt{p}\triangleright\mathtt{m}(S)\cdot\sigma$.

We write γ to denote either a label $\mathtt{p}!\mathtt{m}(S)$ (send message, by rule SQ-Send) or $\mathtt{p}?\mathtt{m}(S)$ (enqueue message, by rule SQ-Recv). ⌟

In Def. 4.3, a session type T transitions by emitting labels representing an internal choice $\mathtt{p}\oplus\mathtt{m}(S)$ or an external choice $\mathtt{p}\&\mathtt{m}(S)$, by rules S-IntC, S-ExtC, and S-Rec. For instance, for the session type $T_{\mathtt{Info}}$ in Example 4.2, these rules yield the transition system in Fig. 9a.

When T is composed with a multi-input queue, rule SQ-Send says that internal choices of T enable a "send" action $\mathtt{p}!\mathtt{m}(S)$. Rule SQ-Deq uses a standard queue congruence $\equiv$ allowing for swapping two queued messages with different senders: this enables the selective dequeuing and consumption of the oldest queued message from each sender. Then, rule SQ-Deq says that the session type can consume the oldest message from $\mathtt{p}$ (from the queue head, via congruence $\equiv$) with an internal action (a.k.a. "τ-action") $\tau(\mathtt{p}\&\mathtt{m}(S))$; this can happen only if T is an external choice that awaits a message $\mathtt{m}$ from $\mathtt{p}$ with payload type S, and the oldest queued message from $\mathtt{p}$ satisfies these conditions. Finally, rule SQ-Recv allows an arbitrary message $\mathtt{p}\triangleright\mathtt{m}(S)$ to be received from the "outside world" and appended to the queue, via a "receive" action $\mathtt{p}?\mathtt{m}(S)$.

▶ **Example 4.4** (LTS semantics in action)**.** Consider the session type from Example 4.2, paired with an initially empty input queue:

$$\mathtt{c}\,\&\,\mathtt{req}(\mathrm{Int}).\ \mathtt{r}\oplus\mathtt{r_req}(\mathrm{Int}).\ \mathtt{d}\oplus\mathtt{d_req}(\mathrm{Int}).\ \mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str}) \langle\!| \epsilon$$

Suppose that $\mathtt{c}$ sends the expected message with label `req` to the end host modelled by this session type. First, the message is moved to the input queue by rule SQ-Recv, via a transition $\xrightarrow{\mathtt{c}?\mathtt{req}(\mathrm{Int})}$, resulting in the following session type with queue:

$$\mathtt{c}\,\&\,\mathtt{req}(\mathrm{Int}).\ \mathtt{r}\oplus\mathtt{r_req}(\mathrm{Int}).\ \mathtt{d}\oplus\mathtt{d_req}(\mathrm{Int}).\ \mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str}) \langle\!| \mathtt{c}\triangleright\mathtt{req}(\mathrm{Int})\cdot\epsilon$$

Since the head of the input queue now contains a message that matches one of the cases in the topmost external choice, rule SQ-Deq enables a transition $\xrightarrow{\tau(\mathtt{c}\&\mathtt{req}(\mathrm{Int}))}$ which consumes the queued message, resulting in:

$$\mathtt{r}\oplus\mathtt{r_req}(\mathrm{Int}).\ \mathtt{d}\oplus\mathtt{d_req}(\mathrm{Int}).\ \mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str}) \langle\!| \epsilon$$

We can then immediately fire the internal choices by applying SQ-Send twice, with transitions $\xrightarrow{\mathtt{r!r_req(Int)}}$ and $\xrightarrow{\mathtt{d!d_req(Int)}}$, resulting in:

$$\mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \epsilon$$

Now, suppose role d sends the expected message with label r_rsp first. We can enqueue the message with rule SQ-Recv and transition $\xrightarrow{\mathtt{d?d_rsp(Str)}}$, leading to the following configuration. Note that the message from d is at the head of the queue, but cannot be consumed yet because it does not match any of the cases in the topmost external choice:

$$\mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \mathtt{d}\triangleright\mathtt{d_rsp}(\mathrm{Str})\cdot\epsilon$$

Once we receive the response message from r and enqueue it with SQ-Recv, we can consume both messages by applying SQ-Deq twice. The first application of SQ-Deq below uses queue congruence $\equiv$ (Def. 4.3) to swap the two messages in the queue (since they have different senders) and bring the one from r to the front, enabling its consumption.

$$\begin{array}{c}\mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \mathtt{d}\triangleright\mathtt{d_rsp}(\mathrm{Str})\cdot\epsilon\\ \xrightarrow{\mathtt{r?r_rsp(Str)}}\quad\text{(SQ-Recv)}\\ \mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \mathtt{d}\triangleright\mathtt{d_rsp}(\mathrm{Str})\cdot\mathtt{r}\triangleright\mathtt{r_rsp}(\mathrm{Str})\cdot\epsilon\\ \xrightarrow{\tau(\mathtt{r}\&\mathtt{r_rsp}(\mathrm{Str}))}\ \xrightarrow{\tau(\mathtt{d}\&\mathtt{d_rsp}(\mathrm{Str}))}\quad\text{(SQ-Deq, SQ-Deq)}\\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \epsilon\end{array}$$

Finally, we send the response to c and end the protocol:

$$\mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \epsilon\xrightarrow{\tau(\mathtt{c!rsp(Str)})}\mathbf{end}\ \langle\!|\ \epsilon\quad\text{(SQ-Send)}$$

⌟

▶ **Example 4.5** (Stuck session types and queues due to bad messages). Rule SQ-Recv in Def. 4.3 allows enqueuing a message that the session type can never consume: this models the case where an unexpected message is delivered to the end host from the surrounding network. E.g., consider this configuration from Example 4.4:

$$\mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \epsilon$$

If the surrounding network now delivers a message with label unexpected from role r, we can enqueue it using SQ-Recv, with transition $\xrightarrow{\mathtt{r?unexpected(Int)}}$ leading to:

$$\mathtt{r}\,\&\,\mathtt{r_rsp}(\mathrm{Str}).\ \mathtt{d}\,\&\,\mathtt{d_rsp}(\mathrm{Str}).\ \mathtt{c}\oplus\mathtt{rsp}(\mathrm{Str})\ \langle\!|\ \mathtt{r}\triangleright\mathtt{unexpected}(\mathrm{Int})\cdot\epsilon$$

This session type with queue is now stuck: it can only proceed by dequeuing a message with label r_rsp from r, but the oldest queued message from r has label unexpected. If more messages are queued, they will not be consumed either. ⌟

We model networks in Def. 4.6 as parallel compositions of roles with an end host behaviour represented as a session type with a multi-input queue.

▶ **Definition 4.6** (Network). *We define a network as:*

$$N\ ::=\ N\parallel N'\ \mid\ \mathtt{p}:\ddot{T}\qquad \textit{where } \mathtt{p} \textit{ does not occur in } \ddot{T}$$

with the following LTS semantics, using the labels $\delta\ ::=\ \mathtt{p}:\gamma\mid\tau(\mathtt{p}:\mathtt{q}\&\mathtt{m}(S))\mid\tau(\mathtt{p}\to\mathtt{q}:\mathtt{m}(S))$ *with* γ *from Def. 4.3: (for brevity we omit the symmetric rules of Net-Par and Net-Comm):*

$$\frac{\ddot{T}\xrightarrow{\gamma}\ddot{T}'}{\mathtt{p}:\ddot{T}\xrightarrow{\mathtt{p}:\gamma}\mathtt{p}:\ddot{T}'}\ \textit{Net-}\gamma\qquad\frac{\ddot{T}\xrightarrow{\tau(\mathtt{q}\&\mathtt{m}(S))}\ddot{T}'}{\mathtt{p}:\ddot{T}\xrightarrow{\tau(\mathtt{p}:\mathtt{q}\&\mathtt{m}(S))}\mathtt{p}:\ddot{T}'}\ \textit{Net-Deq}$$

$$\frac{N_1\xrightarrow{\delta}N_1'}{N_1\parallel N_2\xrightarrow{\delta}N_1'\parallel N_2}\ \textit{Net-Par}\qquad\frac{N_1\xrightarrow{\mathtt{p}:\mathtt{q}!\mathtt{m}(S)}N_1'\quad N_2\xrightarrow{\mathtt{q}:\mathtt{p}?\mathtt{m}(S)}N_2'}{N_1\parallel N_2\xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{m}(S))}N_1'\parallel N_2'}\ \textit{Net-Comm}$$

⌟

In Def. 4.6 above, rules Net-γ and Net-Deq decorate a transition of a session type with input queue $\ddot{T}$ by including the role $\mathtt{p}$ that emitted the transition; specifically, Net-γ is used when $\ddot{T}$ emits or enqueues a message (via rules SQ-Send or SQ-Recv in Def. 4.3), while Net-Deq is used when $\ddot{T}$ internally consumes a queued message (via SQ-Deq in Def. 4.3). Rule Net-Comm says that if in the sub-network N_1 there is $\mathtt{p}$ sending a message to $\mathtt{q}$, and in the sub-network N_2 there is $\mathtt{q}$ that can receive that message, then the network advances with a communication $\tau(\mathtt{p} \to \mathtt{q} : \mathtt{m}(S))$. The rightmost premise of Net-Comm implies that the message from $\mathtt{p}$ is added to $\mathtt{q}$'s input queue, by Net-γ and SQ-Recv in Def. 4.3. Also, by the same rules, *any* N_2 that includes $\mathtt{q}$ can always receive *any* message from *any* $\mathtt{p}$, i.e., rule Net-Comm allows invalid messages to be sent/received between end hosts; see Example 4.7.

▶ **Example 4.7** (Bad messages in an unmonitored network)**.** Consider the following network with roles $\mathtt{p}$, $\mathtt{q}$, and $\mathtt{r}$: (for brevity, we omit the payload types and **end**s)

$$\mathtt{p} : T_{\mathtt{p}} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \epsilon \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon \quad \text{where} \quad T_{\mathtt{p}} = \sum \begin{cases} \mathtt{q} \oplus \mathtt{a} \\ \mathtt{q} \oplus \mathtt{b} \end{cases}; \quad T_{\mathtt{q}} = \mathtt{p} \,\&\, \begin{cases} \mathtt{a}.\mathtt{r} \,\&\, \mathtt{a}' \\ \mathtt{b}.\mathtt{r} \,\&\, \mathtt{b}' \end{cases}; \quad T_{\mathtt{r}} = \mathtt{q} \oplus \mathtt{a}'$$

Here, $T_{\mathtt{p}}$ can send either $\mathtt{a}$ or $\mathtt{b}$ to $\mathtt{q}$. Meanwhile, $T_{\mathtt{q}}$ expects to receive either $\mathtt{a}$ from $\mathtt{p}$ and then $\mathtt{a}'$ from $\mathtt{r}$, or $\mathtt{b}$ from $\mathtt{p}$ and then $\mathtt{b}'$ from $\mathtt{r}$. Instead, $T_{\mathtt{r}}$ just sends $\mathtt{a}'$ to $\mathtt{q}$.

By Def. 4.6 the network could reduce as follows, with $\mathtt{a}$ sent by $\mathtt{p}$ and enqueued by $\mathtt{q}$:

$$\dfrac{\dfrac{\mathtt{p} : T_{\mathtt{p}} \langle\!| \epsilon \xrightarrow{\mathtt{p}:\mathtt{q}!\mathtt{a}} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \qquad \mathtt{q} : T_{\mathtt{q}} \langle\!| \epsilon \xrightarrow{\mathtt{q}:\mathtt{p}?\mathtt{a}} \mathtt{q} : T_{\mathtt{q}} \langle\!| \mathtt{p} \triangleright \mathtt{a}}{\mathtt{p} : T_{\mathtt{p}} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \epsilon \xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{a}())} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \mathtt{p} \triangleright \mathtt{a}}\ \textsc{Net-Comm}}{\mathtt{p} : T_{\mathtt{p}} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \epsilon \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon \xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{a}())} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \mathtt{p} \triangleright \mathtt{a} \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon}\ \textsc{Net-Par}$$

Then, $\mathtt{q}$ can consume the message $\mathtt{a}$ from $\mathtt{p}$, and later enqueue and consume $\mathtt{a}'$ from $\mathtt{r}$: in this case, the network reaches a successful final state where every end host is **end** with an empty queue. Similarly, if $\mathtt{r}$ sends $\mathtt{a}'$ to $\mathtt{q}$ first, and $\mathtt{p}$ sends $\mathtt{a}$ afterwards, then $\mathtt{q}$ can consume both messages (like the last transitions of Example 4.4) reaching success. Therefore, in both cases, $\mathtt{a}'$ from $\mathtt{r}$ is a "good message" for $\mathtt{q}$. However, if $\mathtt{p}$ sends message $\mathtt{b}$ to $\mathtt{q}$, then $T_{\mathtt{q}}$ consumes it and enters the branch where it expects $\mathtt{b}'$ from $\mathtt{r}$ – but $\mathtt{r}$ sends $\mathtt{a}'$ instead:

$$\begin{aligned} \mathtt{p} : T_{\mathtt{p}} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \epsilon \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon &\xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{b}())} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \parallel \mathtt{q} : T_{\mathtt{q}} \langle\!| \mathtt{p} \triangleright \mathtt{b} \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon && \text{(by Net-Comm, Net-Par)} \\ &\xrightarrow{\tau(\mathtt{q}:\mathtt{p}\&\mathtt{b}())} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \parallel \mathtt{q} : \mathtt{r} \,\&\, \mathtt{b}' \langle\!| \epsilon \parallel \mathtt{r} : T_{\mathtt{r}} \langle\!| \epsilon && \text{(by Net-Deq, Net-Par)} \\ &\xrightarrow{\tau(\mathtt{r}\to\mathtt{q}:\mathtt{a}'())} \mathtt{p} : \mathbf{end} \langle\!| \epsilon \parallel \mathtt{q} : \mathtt{r} \,\&\, \mathtt{b}' \langle\!| \mathtt{r} \triangleright \mathtt{a}' \parallel \mathtt{r} : \mathbf{end} \langle\!| \epsilon && \text{(by Net-Comm, Net-Par)} \end{aligned}$$

Therefore, $\mathtt{q}$ cannot consume $\mathtt{r}$'s message and gets stuck – similarly to Example 4.5. Note that in this execution, unlike the cases above, $\mathtt{a}'$ from $\mathtt{r}$ is a "bad message" for $\mathtt{q}$. ⌟

4.2 Monitored Networks

To model the pairing of an end host with a monitor that intercepts all its communications, in Def. 4.8 below we combine a session type with queue $\ddot{T}$ and a generic *monitor* M. For now we only assume that M has an LTS semantics with labels of the form $\mathtt{p}!\mathtt{m}(S)\checkmark$ / $\mathtt{p}?\mathtt{m}(S)\checkmark$ to signal that M accepts the corresponding send/receive action by $\ddot{T}$, and $\mathtt{p}!\mathtt{m}(S)\text{✘}$ / $\mathtt{p}?\mathtt{m}(S)\text{✘}$ to signal that M rejects them. (We present a concrete instantiation of M in Def. 4.10 below.)

▶ **Definition 4.8.** *We define a* ***monitored session type (with input queue)*** *as:*

$$\widehat{T} \;::=\; \lceil \ddot{T} \rceil M$$

where M is a monitor. We also define the monitored session type semantics:

$$\frac{\ddot{T} \xrightarrow{\gamma} \ddot{T}' \qquad M \xrightarrow{\gamma\checkmark} M'}{\lceil \ddot{T} \rceil M \xrightarrow{\gamma\checkmark} \lceil \ddot{T}' \rceil M'}\ \textsc{M-Good} \qquad \frac{\ddot{T} \xrightarrow{\mathtt{p}\oplus\mathtt{m}(S)} \ddot{T}' \qquad M \xrightarrow{\mathtt{p}\oplus\mathtt{m}(S)\times} M'}{\lceil \ddot{T} \rceil M \xrightarrow{\mathtt{p}\oplus\mathtt{m}(S)\times} \lceil \ddot{T}' \rceil M'}\ \textsc{M-BadOut}$$

$$\frac{M \xrightarrow{\mathtt{p}\&\mathtt{m}(S)\times} M'}{\lceil \ddot{T} \rceil M \xrightarrow{\mathtt{p}\&\mathtt{m}(S)\times} \lceil \ddot{T} \rceil M'}\ \textsc{M-BadIn} \qquad \frac{\ddot{T} \xrightarrow{\tau(\mathtt{p}\&\mathtt{m}(S))} \ddot{T}'}{\lceil \ddot{T} \rceil M \xrightarrow{\tau(\mathtt{p}\&\mathtt{m}(S))} \lceil \ddot{T}' \rceil M}\ \textsc{M-Dequeue}$$

⌟

By rule M-Good in Def. 4.8, the pair $\lceil \ddot{T} \rceil M$ performs a transition $\gamma\checkmark$ if M explicitly accepts the send/receive action γ emitted by $\ddot{T}$. Rule M-BadOut says that M can reject and drop a message sent by $\ddot{T}$. Rule M-BadIn says that M can reject and drop an incoming message before it lands in $\ddot{T}$'s input queue. By rule M-Dequeue, M cannot interfere with the internal action that $\ddot{T}$ performs when consuming a message from its input queue. In Def. 4.9 below we monitor networks (Def. 4.6), by adding a monitor to each end host.

▶ **Definition 4.9** (Monitored Network)**.** *We define a monitored network as:*

$$\widehat{N} \;::=\; \widehat{N} \parallel \widehat{N}' \;\mid\; \mathtt{p} : \widehat{T} \qquad \textit{where } \mathtt{p} \textit{ does not occur in } \widehat{T}$$

with the following semantics, using the labels $\eta ::= \mathtt{p} : \gamma\checkmark \mid \mathtt{p} : \gamma\times \mid \tau(\mathtt{p} : \mathtt{q}\&\mathtt{m}(S)) \mid \tau(\mathtt{p} \to \mathtt{q} : \mathtt{m}(S))$ (for brevity we omit the symmetric rules of MNet-Par and MNet-Comm):

$$\frac{\widehat{T} \xrightarrow{\gamma\checkmark} \widehat{T}'}{\mathtt{p} : \widehat{T} \xrightarrow{\mathtt{p}:\gamma\checkmark} \mathtt{p} : \widehat{T}'}\ \textsc{MNet-}\gamma\textsc{-Good} \qquad \frac{\widehat{T} \xrightarrow{\gamma\times} \widehat{T}'}{\mathtt{p} : \widehat{T} \xrightarrow{\mathtt{p}:\gamma\times} \mathtt{p} : \widehat{T}'}\ \textsc{MNet-}\gamma\textsc{-Bad} \qquad \frac{\widehat{N}_1 \xrightarrow{\eta} \widehat{N}_1'}{\widehat{N}_1 \parallel \widehat{N}_2 \xrightarrow{\eta} \widehat{N}_1' \parallel \widehat{N}_2}\ \textsc{MNet-Par}$$

$$\frac{\widehat{T} \xrightarrow{\tau(\mathtt{q}\&\mathtt{m}(S))} \widehat{T}'}{\mathtt{p} : \widehat{T} \xrightarrow{\tau(\mathtt{p}:\mathtt{q}\&\mathtt{m}(S))} \mathtt{p} : \widehat{T}'}\ \textsc{MNet-Deq} \qquad \frac{\widehat{N}_1 \xrightarrow{\mathtt{p}:\mathtt{q}!\mathtt{m}(S)\checkmark} \widehat{N}_1' \qquad \widehat{N}_2 \xrightarrow{\mathtt{q}:\mathtt{p}?\mathtt{m}(S)\checkmark} \widehat{N}_2'}{\widehat{N}_1 \parallel \widehat{N}_2 \xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{m}(S))} \widehat{N}_1' \parallel \widehat{N}_2'}\ \textsc{MNet-Comm}$$

⌟

In Def. 4.9, each end host is modelled as a role p with a session type (with an input queue) representing the end host behaviour, equipped with a monitor. Rules MNet-γ-Good and MNet-γ-Bad annotate accepted and rejected actions (with labels $\gamma\checkmark$ and $\gamma\times$ from Def. 4.8) with the end host role p where such actions occurred. The rule MNet-Comm is different from Net-Comm in Def. 4.6, because it only allows communications between two end hosts if their respective monitors accept their outgoing and incoming message; this is because the transitions in the premises of rule MNet-Comm (denoting the acceptance of a message send and enqueuing) can only be fired via rule MNet-γ-Good.

4.3 Synthesising Network-Layer Monitors from Session Types

We now provide a concrete instantiation of monitor M from Def. 4.8: in Def. 4.10 we formalise how to synthesise a network-layer *monitor state* $[\![T]\!]$ from a session type T. In §3.1 (Fig. 9) we anticipated that the monitor state machine $[\![T]\!]$ differs from that of T. This is because $[\![T]\!]$ has to meet several non-trivial requirements:

R1. $[\![T]\!]$ must accept all messages that an end host implementing T may send/receive to/from the network, depending on T's state;

R2. $[\![T]\!]$ should reject invalid messages that an end host implementing T should *not* send/receive, again depending on T's state;

R3. $[\![T]\!]$ may receive messages from different senders in an order that does not match T's expectations (due to the network semantics),[9] and yet, $[\![T]\!]$ must accept the valid ("good") messages while still rejecting the invalid ("bad") ones;

R4. $[\![T]\!]$ must decide whether to accept or reject a message *immediately*, *without buffering*, to accommodate the limited memory and processing power of most network devices.

▶ **Definition 4.10** (Session-type-based network monitor). *We write* $[\![T]\!]$ *to represent the state of a monitor* M *based on a session type* T*, with semantics given by the following rules:*

$$\frac{k \in I}{\left[\!\left[\sum_{i\in I} \mathtt{p}_i \oplus \mathtt{m}_i(S_i).T_i\right]\!\right] \xrightarrow{\mathtt{p}_k ! \mathtt{m}_k(S_k)\,\checkmark} [\![T_k]\!]}\ \textsc{STMon-IntC} \qquad \frac{k \in I}{\left[\!\left[\mathtt{p} \& _{i\in I} \mathtt{m}_i(S_i).T_i\right]\!\right] \xrightarrow{\mathtt{p} ? \mathtt{m}_k(S_k)\,\checkmark} [\![T_k]\!]}\ \textsc{STMon-ExtC}$$

$$\frac{\exists\ \textit{maximal}\ \emptyset \neq K \subseteq I, \mathtt{q}, \mathtt{m}, S: \quad \forall i \in I: \mathtt{q} \neq \mathtt{p}_i \quad \forall k \in K: \exists T'_k: [\![T_k]\!] \xrightarrow{\mathtt{q}?\mathtt{m}(S)\,\checkmark} [\![T'_k]\!]}{\left[\!\left[\sum_{i\in I} \mathtt{p}_i \oplus \mathtt{m}_i(S_i).T_i\right]\!\right] \xrightarrow{\mathtt{q}?\mathtt{m}(S)\,\checkmark} \left[\!\left[\sum_{k\in K} \mathtt{p}_k \oplus \mathtt{m}_k(S_k).T'_k\right]\!\right]}\ \textsc{STMon-IntPfx}$$

$$\frac{\exists\ \textit{maximal}\ \emptyset \neq K \subseteq I, \mathtt{q}, \mathtt{m}, S: \quad \mathtt{q} \neq \mathtt{p} \quad \forall k \in K: \exists T'_k: [\![T_k]\!] \xrightarrow{\mathtt{q}?\mathtt{m}(S)\,\checkmark} [\![T'_k]\!]}{\left[\!\left[\mathtt{p} \& _{i\in I} \mathtt{m}_i(S_i).T_i\right]\!\right] \xrightarrow{\mathtt{q}?\mathtt{m}(S)\,\checkmark} \left[\!\left[\mathtt{p} \& _{k\in K} \mathtt{m}_k(S_k).T'_k\right]\!\right]}\ \textsc{STMon-ExtPfx}$$

$$\frac{[\![T\{\mathtt{t} \mapsto \mu \mathtt{t}.T\}]\!] \xrightarrow{\gamma\,\checkmark} [\![T']\!]}{[\![\mu \mathtt{t}.T]\!] \xrightarrow{\gamma\,\checkmark} [\![T']\!]}\ \textsc{STMon-Rec} \qquad \frac{[\![T]\!] \xrightarrow{\gamma\,\checkmark} \not\rightarrow}{[\![T]\!] \xrightarrow{\gamma\,\times} [\![T]\!]}\ \textsc{STMon-Bad}$$

We write $[\![T \langle\!| \sigma]\!]$ *to represent the monitor state defined as follows:*

$$[\![T \langle\!| \sigma]\!] = [\![T']\!] \qquad \textit{if and only if} \qquad [\![T]\!] \xrightarrow{?\sigma\,\checkmark} [\![T']\!]$$

where $[\![T]\!] \xrightarrow{?(\mathtt{p}_i \triangleright \mathtt{m}_1(T_1) \cdot \cdots \cdot \mathtt{p}_n \triangleright \mathtt{m}_n(T_n))\,\checkmark} [\![T']\!]$ *iff* $[\![T]\!] \xrightarrow{\mathtt{p}_1 ? \mathtt{m}_1(S_1)\,\checkmark} \cdots \xrightarrow{\mathtt{p}_n ? \mathtt{m}_n(S_n)\,\checkmark} [\![T']\!]$. ⌟

By rule STMon-IntC in Def. 4.10, if T is an internal choice, then $[\![T]\!]$ accepts the corresponding send actions and updates its state. Dually, by rule STMon-ExtC, if T is an external choice, then $[\![T]\!]$ accepts the corresponding receive actions and updates its state. This reflects requirements R 1 and R 4.

Rules STMon-IntPfx and STMon-ExtPfx allow $[\![T]\!]$ to accept an incoming message $\mathtt{m}$ with payload type S from role $\mathtt{q}$, even if the shape of T does not expect a message from $\mathtt{q}$ right now. This is necessary to satisfy requirements R 3 and R 4. By the premises of these rules, acceptance is allowed only if role $\mathtt{q}$ is *not* an immediate recipient/sender in the internal/external choice T, and at least one monitor $[\![T_k]\!]$ (where T_k is a continuation of T) can indeed accept that message by firing a transition $[\![T_k]\!] \xrightarrow{\mathtt{q}?\mathtt{m}(S)\,\checkmark} [\![T'_k]\!]$. If these conditions hold, then after accepting $\mathtt{q}?\mathtt{m}(S)$, rules STMon-IntPfx and STMon-ExtPfx "prune" T by removing all the branches that, if taken, could *not* possibly accept $\mathtt{q}?\mathtt{m}(S)$ in their future transitions. More precisely, $[\![T]\!]$ performs the same accepting transition $\mathtt{q}?\mathtt{m}(S)\,\checkmark$ and becomes $[\![T']\!]$, where T' has the same shape as T, except that:

1. T' keeps all (and only) the branches of T (indexed by the maximal set $K \subseteq I$) that could accept $\mathtt{q}?\mathtt{m}(S)$ in their future transitions; and
2. The continuation of each kept branch is reduced to T'_k (for $k \in K$).

Note that these rules can fire only if $K \neq \emptyset$: there must therefore be at least one branch of T that can accept $\mathtt{q}?\mathtt{m}(S)$ in its future transitions.

[9] In §3.3 we also addressed the issue of out-of-order delivery of messages *from the same sender*, which is orthogonal and handled by protocols like TCP.

Rule STMon-Rec unfolds recursion. Rule STMon-Bad rejects any send/receive action that $[\![T]\!]$ does not explicitly accept, per requirements R 2 and R 4. Finally, $[\![T \langle\!| \sigma]\!]$ represents the monitor state obtained by feeding all messages in σ as inputs to $[\![T]\!]$, which must accept all of them: i.e., $[\![T \langle\!| \sigma]\!]$ is undefined if $[\![T]\!]$ does not accept some message in σ.

Examples. To illustrate how our session monitors work, we present three examples:

- Example 4.11 shows how a monitor $[\![T]\!]$ can accept messages that arrive in an order different from T's expectations, and how doing so restricts the inputs and outputs it will accept next.
- Example 4.12 revisits Example 4.4 to track how a monitor evolves alongside the end host's session type and input queue.
- Example 4.13 shows that some session types yield infinite-state monitors under Def. 4.10, which cannot be represented using a finite number of states in P4 (§3).
- A further monitor-reduction example is available in [46, Example A.1].

▶ **Example 4.11.** Consider the type $T_{\mathtt{q}}$ from Example 4.7: (we omit payload types)

$$T_{\mathtt{q}} \;=\; \mathtt{p} \,\&\, \begin{cases} \mathtt{a}.\mathtt{r} \,\&\, \mathtt{a}'.\mathbf{end} \\ \mathtt{b}.\mathtt{r} \,\&\, \mathtt{b}'.\mathbf{end} \end{cases}$$

By Def. 4.10, the corresponding monitor $[\![T_{\mathtt{q}}]\!]$ can immediately accept not only the two top-level messages from $\mathtt{p}$, but also the successive messages from $\mathtt{r}$ – which appear later in $T_{\mathtt{q}}$, but may be delivered earlier by the surrounding network. For the top-level inputs we have:

$$[\![T_{\mathtt{q}}]\!] \xrightarrow{\mathtt{p\&a}()\,\checkmark} [\![\mathtt{r} \,\&\, \mathtt{a}'.\mathbf{end}]\!] \quad \text{and} \quad [\![T_{\mathtt{q}}]\!] \xrightarrow{\mathtt{p\&b}()\,\checkmark} [\![\mathtt{r} \,\&\, \mathtt{b}'.\mathbf{end}]\!] \qquad \text{(by STMon-ExtC in Def. 4.10)}$$

Notice that the message sent by $\mathtt{p}$ restricts what the monitor $[\![T_{\mathtt{q}}]\!]$ accepts from $\mathtt{r}$ afterwards. If $[\![T_{\mathtt{q}}]\!]$ is deployed in the network of Example 4.7 to monitor end host $\mathtt{q}$, then, if $\mathtt{p}$ sends $\mathtt{a}$, the monitor will accept $\mathtt{a}'$ from $\mathtt{r}$ (which is a "good" message in this state); instead, if $\mathtt{p}$ sends $\mathtt{b}$, the monitor will reject $\mathtt{a}'$ from $\mathtt{r}$ (which is a "bad" message in this state).

Notably, the monitor $[\![T_{\mathtt{q}}]\!]$ can also immediately accept the messages from $\mathtt{r}$. The transitions $\mathtt{r}?\mathtt{a}'\,\checkmark$ and $\mathtt{r}?\mathtt{b}'\,\checkmark$ are fired by the following derivations:

$$\dfrac{\dfrac{}{[\![\mathtt{r} \,\&\, \mathtt{a}'.\mathbf{end}]\!] \xrightarrow{\mathtt{r}?\mathtt{a}'\,\checkmark} [\![\mathbf{end}]\!]}\ \text{STMon-ExtC}}{[\![T_{\mathtt{q}}]\!] \xrightarrow{\mathtt{r}?\mathtt{a}'\,\checkmark} [\![\mathtt{p} \,\&\, \mathtt{a}.\mathbf{end}]\!]}\ \text{STMon-ExtPfx} \qquad \dfrac{\dfrac{}{[\![\mathtt{r} \,\&\, \mathtt{b}'.\mathbf{end}]\!] \xrightarrow{\mathtt{r}?\mathtt{b}'\,\checkmark} [\![\mathbf{end}]\!]}\ \text{STMon-ExtC}}{[\![T_{\mathtt{q}}]\!] \xrightarrow{\mathtt{r}?\mathtt{b}'\,\checkmark} [\![\mathtt{p} \,\&\, \mathtt{b}.\mathbf{end}]\!]}\ \text{STMon-ExtPfx}$$

Observe that the message $\mathtt{a}'$ (resp. $\mathtt{b}'$) from $\mathtt{r}$ causes rule STMon-ExtPfx to "prune" the session type $T_{\mathtt{q}}$ in the monitor state, only keeping the branch where $\mathtt{a}$ (resp. $\mathtt{b}$) from $\mathtt{p}$ can be received. Therefore, if $[\![T_{\mathtt{q}}]\!]$ is deployed in the network of Example 4.7 to monitor end host $\mathtt{q}$, it will accept $\mathtt{a}'$ from $\mathtt{r}$ even before $\mathtt{p}$ sends $\mathtt{a}$ or $\mathtt{b}$ – because in this state it is still possible for the end host to consume $\mathtt{a}'$ without getting stuck. Then, after accepting $\mathtt{a}'$ from $\mathtt{r}$:

1. The monitor will accept $\mathtt{a}$ from $\mathtt{p}$ – which is a "good" message in this state, because $T_{\mathtt{q}}$ can consume $\mathtt{a}$ from $\mathtt{p}$ and then $\mathtt{a}'$ from $\mathtt{r}$ from the end host's input queue. However,
2. The monitor will reject $\mathtt{b}$ from $\mathtt{p}$ – which is a "bad" message in this state, because $T_{\mathtt{q}}$ cannot consume $\mathtt{b}$ from $\mathtt{p}$ and then $\mathtt{a}'$ from $\mathtt{r}$ (as shown at the end of Example 4.7).

This strategy for accepting messages is necessary because, depending on the overall multiparty interaction, messages from $\mathtt{r}$ may be delivered before those from $\mathtt{p}$. This phenomenon is further illustrated in Example 4.12 below. ⌟

▶ **Example 4.12.** Consider the session type $T_{\texttt{Info}}$ from Example 4.2 (for the `Info` role in Fig. 4): its LTS is shown in Fig. 9a. Consider also the example execution of $T_{\texttt{Info}}$ (with an input queue) in Example 4.4. We now instrument $T_{\texttt{Info}}$ and an empty input queue with a monitor $[\![T_{\texttt{Info}}]\!]$, visualised in Fig. 9b, obtaining $\lceil T_{\texttt{Info}} \langle\!| \epsilon \rceil [\![T_{\texttt{Info}}]\!]$ (by Def. 4.8); we explain how their respective states change as they send/receive messages, according to Def. 4.8.

In Fig. 9, the session type $T_{\texttt{Info}}$ and its monitor $[\![T_{\texttt{Info}}]\!]$ begin in states s0 and m0 respectively. At this point, $[\![T_{\texttt{Info}}]\!]$ will *only* allow the message $\texttt{c}?\texttt{req}(\text{Int})$ from the `Client` (represented by `c`) to go through. Once the message arrives, $[\![T_{\texttt{Info}}]\!]$ will accept it by progressing to state m1 (by STMon-ExtC). As part of accepting the message, the monitor forwards it to the end host input queue, and then the session type $T_{\texttt{Info}}$ consumes it (by SQ-Recv and SQ-Deq), reaching state s1. The session type can then immediately progress to state s2, then s3, by sending $\texttt{r}!\texttt{r_req}(\text{Int})$ and $\texttt{d}!\texttt{d_req}(\text{Int})$ to `Review` (role `r`) and `Details` (role `d`) respectively. Let's assume that the monitor forwards both of these messages before it sees a response, progressing to state m2, then m4.

Now, the session type (now in state s3) expects a response from `r` and then from `d`, but there is no guarantee that the responses will be delivered in this exact order. The monitor (now in state m2) accounts for this. Suppose that the monitor receives $\texttt{d}?\texttt{d_rsp}(\text{Str})$ as the first response. The monitor accepts the message, and progresses to state m6 while forwarding the message. The session type, however, does not progress immediately, but remains in state s3 as the message from `d` in its queue does not match any of the branches in its external choice. (See the execution in Example 4.4.)

Eventually, the monitor also receives the response message $\texttt{r}?\texttt{r_rsp}(\text{Str})$ from `r`, and progresses to state m7 while forwarding the message to the session type's input queue – which can then finally dequeue both messages from `r` and `d`, proceeding to state s4, then s5.

Finally, the session type (now in state s5) sends the message $\texttt{c}!\texttt{rsp}(\text{Str})$ to `c` and progresses to state s6; the monitor (now in state m7) accepts the outgoing message and progresses to state m8, at which point the protocol has finished. ⌟

▶ **Example 4.13** (On unmonitorable session types)**.** Consider the session type $\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}$ (for brevity, we omit the payload types). By STMon-ExtC and STMon-Rec, a monitor with this session type in its state can transition by receiving from `q`. Moreover, the same monitor can transition by receiving from `p`, with the following derivation:

$$\dfrac{\dfrac{\dfrac{}{[\![\texttt{p}\&\texttt{b}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!]}\ \text{STMon-ExtC}}{[\![\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\texttt{q}\&\texttt{a}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!]}\ \text{STMon-ExtPfx}}{[\![\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\texttt{q}\&\texttt{a}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!]}\ \text{STMon-Rec}$$

The monitor could then accept an incoming message `a` from `q`, and return to its original state. However, the monitor can also accept the next input from `p`:

$$\dfrac{\dfrac{\dfrac{\vdots\ \textit{(Same derivation above)}}{[\![\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\texttt{q}\&\texttt{a}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!]}\ \text{STMon-Rec}}{[\![\texttt{q}\&\texttt{a}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\texttt{q}\&\texttt{a}.\texttt{q}\&\texttt{a}.\mu\texttt{t}.\texttt{q}\&\texttt{a}.\texttt{p}\&\texttt{b}.\texttt{t}]\!]}\ \text{STMon-ExtPfx}$$

We can repeat this transition to accept more inputs from `p`, each time reaching a new monitor state that expects more inputs from `q`: $\ldots \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} [\![\texttt{q}\&\texttt{a}.\texttt{q}\&\texttt{a}.\texttt{q}\&\texttt{a}.\texttt{q}\&\texttt{a}.\ \ldots]\!] \xrightarrow{\texttt{p}?\texttt{b}\,\checkmark} \ldots$ Consequently, the LTS of this session-type monitor has infinitely many states.

Our monitor synthesis implementation (§3) rejects session types such as the one in this example, because the monitor state machine is constrained by the (often limited) amount of storage available in network hardware. To avoid infinite-state monitors, the session types being monitored cannot receive unbounded inputs from multiple roles. Many communication protocols involve "request-response" patterns that keep our monitors finite-state, including all the examples we evaluate in §5. ⌟

4.4 Soundness of Session-Types-Based Network Monitoring

A non-negotiable feature of session-type-based monitors from Def. 4.10 is *soundness*: this means that monitors must not reject "good" messages – i.e., monitors must not produce false positives and interfere with a well-behaved network. We formalise this intuition by considering a monitored network $\widehat{N}$ where all monitors $[\![T_{\mathtt{p}}]\!]$ (for all roles $\mathtt{p}$ in $\widehat{N}$) are based on session types $T_{\mathtt{p}}$ that are mutually compatible, and each end host for role $\mathtt{p}$ behaves according to $T_{\mathtt{p}}$. In Theorem 4.18 we show that the monitors in such $\widehat{N}$ are *transparent*: they never disrupt communications between well-behaved hosts.

We now develop the technical machinery for this result. In Def. 4.14 we define a *consistent instrumentation* where each end host is given a monitor matching the end host specification.

▶ **Definition 4.14** (Consistent Instrumentation of a Network). *Given a network N, we define its* consistent monitor instrumentation $\mathrm{mon}(N)$ *as:*

$$\mathrm{mon}\left(\Vert_{i\in I}\ \mathtt{p}_i : \ddot{T}_i\right) = \Vert_{i\in I}\ \mathtt{p}_i : (\lceil \ddot{T}_i \rceil\, [\![\ddot{T}_i]\!])$$

⌟

For an arbitrary N, the instrumented network $\mathrm{mon}(N)$ may reject messages if the underlying session types are not "compatible" with each other, e.g., some $\mathtt{p}$ may send to $\mathtt{q}$ a message that $\mathtt{q}$ does not expect. For instance, if N is the network in Example 4.7, then $\mathrm{mon}(N)$ would reject messages as shown in Example 4.11.

To prove monitor soundness, we must ensure that monitored session types are compatible: we require *output-liveness* as in Def. 4.15 below. Our output-liveness is a weaker variant of the *typing context liveness* property adopted in many session typing papers [63, 4, 21, 58, 70]: like the standard liveness definition, we require that messages sent by a participant are eventually consumed by the intended recipient (assuming fair scheduling) – but unlike the standard definition, we do *not* require that a participant awaiting a message will eventually receive one. In other words, our Def. 4.15 does *not* allow a network to have orphan messages that are sent and queued but never consumed – but it allows a network to have participants that wait forever to receive messages which are never sent. To formalise this, Def. 4.15 uses *paths*, i.e., possible network executions; a path is *fair* if it eventually allows all enabled communications between roles (item 1) and dequeuing actions (item 2) to occur;[10] a path is *output-live* if every queued message is eventually consumed by its intended recipient.

▶ **Definition 4.15** (Output-Live Session Type Networks, adapted from [21, Def. 4.7]). *A* network path *is a possibly infinite sequence of network configurations $(N_i)_{i\in I}$, where $I = \{0, 1, \ldots\}$ is a set of consecutive natural numbers and, $\forall i \in I$, $N_i \xrightarrow{\tau(\gamma_i)} N_i'$. We say that* a network path is fair *iff, $\forall i \in I$:*

[10] Note that in item 1 of Def. 4.15, the existence of $\mathtt{q}, \mathtt{m}, S$ establishes that $\mathtt{p}$ is ready to send some message (with an internal choice) that a recipient $\mathtt{q}$ is ready to enqueue – while $\mathtt{q}', \mathtt{m}', S'$ are the actual recipient and message selected by $\mathtt{p}$ in this execution path.

1. *if* $N_i \xrightarrow{\tau(\mathtt{p}\to\mathtt{q}:\mathtt{m}(S))}$, *then* $\exists k \in I$ *such that* $k \geq i$ *and* $N_k \xrightarrow{\tau(\mathtt{p}\to\mathtt{q}':\mathtt{m}'(S'))} N_{k+1}$;
2. *if* $N_i \xrightarrow{\tau(\mathtt{p}:\mathtt{q}\&\mathtt{m}(S))}$, *then* $\exists k \in I$ *such that* $k \geq i$ *and* $N_k \xrightarrow{\tau(\mathtt{p}:\mathtt{q}\&\mathtt{m}(S))} N_{k+1}$.

We say that a network path is output-live *iff, taking any* $i \in I$ *and letting* $N_i = \Vert_{j\in J}\, \mathtt{p}_j : (T_j \langle\!\langle\, \sigma_j)$, $\forall j \in J$ *we have that if* $\sigma_j \equiv \mathtt{q}\triangleright\mathtt{m}(S)\cdot\sigma'$, *then* $\exists k \in I$ *such that* $k \geq i$ *and* $N_k \xrightarrow{\tau(\mathtt{p}_j:\mathtt{q}\&\mathtt{m}(S))} N_{k+1}$.

We say that N is output-live *if every fair path beginning with* N *is output-live.* ⌟

▶ **Example 4.16** (Output-live networks). Consider the network N in Example 4.7: N is not output-live, because it has a fair path where $\mathtt{r}$ sends $\mathtt{a}'$ to $\mathtt{q}$, $\mathtt{p}$ sends $\mathtt{b}$ to $\mathtt{q}$, hence $\mathtt{q}$ cannot consume the queued message $\mathtt{a}'$ from $\mathtt{r}$. In contrast, the network N' obtained by replacing the session type of $\mathtt{p}$ with $T'_{\mathtt{p}} = \mathtt{q}\oplus\mathtt{a}.\mathbf{end}$ is output-live: in every fair path of N' every queued message is eventually consumed. Also, all the examples evaluated in §5 are output-live. ⌟

To state our monitoring soundness result, in Def. 4.17 we define when two networks have equivalent *internal* behaviour, i.e., communicate and consume messages in the same way.

▶ **Definition 4.17** (Internal Bisimulation). *Let* ϕ *be an annotation to distinguish* τ*-labels of the form* $\tau(\phi)$. *We say that* $\mathcal{R}$ is an internal bisimulation relation *iff, whenever* $(s,t) \in \mathcal{R}$,

1. *if* $s \xrightarrow{\tau(\phi)} s'$, *then* $\exists t'$ *such that* $t \xrightarrow{\tau(\phi)} t'$ *and* $(s',t') \in \mathcal{R}$;
2. *if* $t \xrightarrow{\tau(\phi)} t'$, *then* $\exists s'$ *such that* $s \xrightarrow{\tau(\phi)} s'$ *and* $(s',t') \in \mathcal{R}$.

We say s *and* t *are* internally bisimilar, *written* $s \overset{\tau}{\sim} t$, *iff there is an internal bisimulation* $\mathcal{R}$ *such that* $(s,t) \in \mathcal{R}$. ⌟

The last ingredient to ensure monitoring correctness is a further *half-duplex* assumption to control monitor state-space size. Intuitively, a network N is *half-duplex* if, for any two roles $\mathtt{p}$ and $\mathtt{q}$ in N, data can flow only in one direction at a time, i.e., if $\mathtt{p}$ sent a message to $\mathtt{q}$, then $\mathtt{q}$ must consume that message before sending another message to $\mathtt{p}$ (and *vice versa*). In other words, $\mathtt{p}$ and $\mathtt{q}$ can only communicate by "taking turns" – thus, the input queue of $\mathtt{p}$ can contain a message from $\mathtt{q}$ only if the input queue of $\mathtt{q}$ does *not* contain any message from $\mathtt{p}$. All the examples we evaluate in §5 are half-duplex. *(For the formal definition of half-duplex and an example showing why we need it, see [46, Def. A.2 and Example A.3].)*

We now have all the ingredients to state and prove that our monitors are *sound* by runtime verification standards [2], i.e., they have no false positives.[11] In our setting, this means they never misclassify a good message as bad and never interfere with well-behaved end hosts, if the implemented protocol is output-live and half-duplex. *(Proof in [46, §A.1].)*

▶ **Theorem 4.18** (Monitor Soundness). *If* N *is output-live and half-duplex, then* $N \overset{\tau}{\sim} \mathrm{mon}(N)$.

▶ Remark 4.19 (On determining output-liveness and half-duplex properties). Output-liveness (Def. 4.15) is generally undecidable, since two session types with unbounded queues can encode a Turing machine [3, Theorem 2.5]. It can, however, be guaranteed by decidable approximations such as bounded model checking or projection from a global type [43, 47]. Similar techniques can be used to ensure half-duplex execution [66]. These checks are orthogonal to this work. The protocols we evaluate in §5 are output-live and half-duplex, with bounded queue sizes, so they have finite LTSs and are amenable to model checking.

[11] Another desirable property for monitors is *completeness*, i.e., having no false negatives. Here we focus on soundness because it is non-negotiable, and completeness may not be achievable together with soundness: we discuss these issues in §7.

Table 1 The 10 test case protocols on which we evaluated NEST. For each one of them we report the number of participants, and whether the session type includes branching and/or loops.

Test case	Participants	Branching	Loops	Description
BookInfo [34]	5			match a review to a book name
Store management	7	✓	✓	online ordering service
VPN	4	✓	✓	authenticate communication
Stateful firewall	2	✓	✓	traffic filtering
DNS [42]	5			DNS resolver server
Auction protocol	3	✓	✓	two-buyer auction protocol
CDN [42]	4			content distribution network
SIP [64]	3	✓		session initiation protocol over proxy
POP3 [52][61]	2	✓	✓	client sends multiple queries to a server
Multiplayer game	4	✓	✓	turn-based game

▶ Remark 4.20 (On the monitor rejection strategy). The particular rejection strategy for session-type monitors does not affect the soundness Theorem 4.18, which only concerns accepted behaviour. Concretely, Def. 4.10 says that the monitor's verdict is not persistent: if a monitor rejects a message, then it can still accept a subsequent valid message. Theorem 4.18 would still hold, for example, if rule STMon-Bad in Def. 4.10 always moved $[\![T]\!]$ to $[\![\mathbf{end}]\!]$ after an invalid send/receive, making the rejection verdict persistent and blocking any subsequent message to/from the end host. Indeed, NEST's TCP-oriented monitors (described in §3.3) use persistent verdicts: they block all end host communications (by dropping the whole TCP connection) when the end host sends a "bad" message (as we show in §5.2).

5 Empirical Evaluation

In this section, we evaluate NEST's correctness and effectiveness by addressing these questions:

Q1. Can NEST generate monitors for non-trivial multiparty protocols?

Q2. Do NEST-generated monitors accommodate correct communication without interference, while rejecting incorrect messages even in the presence of packet loss, duplication, and reordering (when using the TCP-oriented monitors described in §3.3)?

Q3. Does NEST support monitoring multiple concurrent sessions?

We first describe test cases and setup (§5.1), then analyse a representative case (§5.2), and finally report aggregate monitoring statistics (§5.3).

5.1 Test Cases and Evaluation Setup

Methodology. To address question Q 1, we selected non-trivial test cases based on real-world multiparty protocols, with varying numbers of participants and branching/looping structures (Table 1). We describe each test case in [46, §B], together with the local session type of each participant. In each test case, participant behaviour is specified as a session type, and the full system is output-live and half-duplex (see [46, Def. A.2] and Def. 4.15). This guarantees the formal preconditions for sound monitoring (Theorem 4.18) and provides the basis for empirically evaluating question Q 2 – for which here we also consider networks with packet loss, duplication, and reordering (that are not formally covered by Theorem 4.18).

Evaluation Setup. For each test case in Table 1, we set up a simulated network in which end hosts (i.e., multiparty-protocol participants) communicate via P4-enabled border switches, as in Fig. 1. We use Mininet [51], which allows us to simulate networks with different topologies,

end hosts, and switch configurations. The simulated network includes nodes running BMv2 [13], a P4-enabled software switch. These switches perform regular forwarding when monitoring is disabled, and deploy/run our NEST-generated monitors to evaluate question Q 1.[12]

To evaluate question Q 2, for each test case in Table 1 we provide correct and faulty participant implementations and assess whether NEST-generated monitors accept or reject packets as expected. Each variant is implemented in Python and executed on Mininet using the NEST-generated API (§3.4).

To assess whether question Q 2 can be answered positively under different transports, we implement each scenario in Table 1 with both UDP and TCP communication. For TCP, we evaluate the behaviour of our TCP-oriented monitors (§3.3) both on a perfectly reliable network, and on an unreliable network with packet loss, duplication, and delay: specifically, we configure Mininet end hosts to drop 1% of incoming packets, duplicate 1% of all outgoing packets, and delay the sending of outgoing packets by a variable amount (up to 100 ms). The delay perturbs packet ordering both across senders and for packets with the same sender.

To evaluate question Q 3, we run multiple concurrent sessions per test case (typically 10 to 50). All experiments were conducted on a machine with an 8-core, 3 GHz CPU and 32 GB of RAM, running Ubuntu 22.04.

5.2 Assessing the Correctness of NEST-Generated Session Monitors

To illustrate our evaluation of question Q 2 (i.e., whether NEST-generated monitors accept/reject messages correctly), we focus on one of the 10 test cases in Table 1: the BookInfo protocol [34], our running example from §1. We applied the same assessment to every test case in Table 1 and observed similar results, so the analysis below is representative. We also briefly report results for the VPN test case, which is structurally richer than BookInfo and covers all session-type features (branching and nested loops). Full details for the other test cases are available in [46, §B].

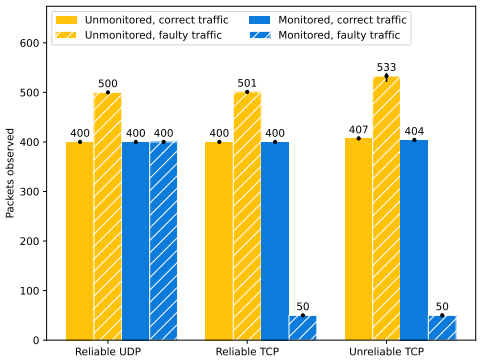
Fig. 12 reports cumulative packets received across all hosts under different configurations (UDP or TCP, reliable or unreliable TCP networks, faulty or correct hosts, with or without runtime monitors). The bars show the median packet counts over 5 runs; the black dot/line on top of each bar shows the maximum and minimum counts (most noticeable in Fig. 12a).

The "correct traffic" bars in Fig. 12a (solid yellow and blue columns) represent BookInfo runs where each end host correctly implements the session protocol. NEST-generated monitors do not reject any packets: the same number of packets is observed in both monitored (blue) and unmonitored (yellow) networks. This is consistent with our soundness Theorem 4.18.

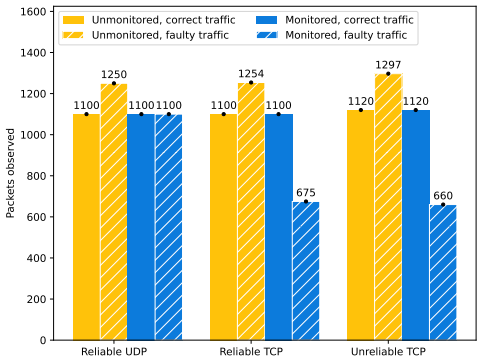
In BookInfo configurations with "faulty traffic", the end host for participant `Info` runs a program that does *not* conform to the expected protocol because it sends incorrect messages to other hosts (specifically, to `Review` and `Details`, see Fig. 13). Key observations:

- The total number of packets with faulty traffic on unmonitored networks (yellow hatched columns) is higher than that for correct traffic (yellow solid columns), because all faulty packets reach their destination end host.

[12] A drawback of this setup is that we cannot perform meaningful performance evaluations, as BMv2 does not reflect the performance characteristics of P4-enabled hardware. Careful hardware experiments would require significant engineering efforts that are orthogonal to the main contributions of this paper; still, previous work [30, 38] suggests that, if the NEST-generated P4 monitors are compiled to a hardware platform without exceeding the available resources, then they will run with little to no overhead, no matter how much traffic the device is processing, up to its limit.



(a) Packets observed for the BookInfo example over different network configurations.



(b) Packets observed for the VPN example over different network configurations.

Figure 12 Incoming packets observed across all hosts, in different examples and configurations.

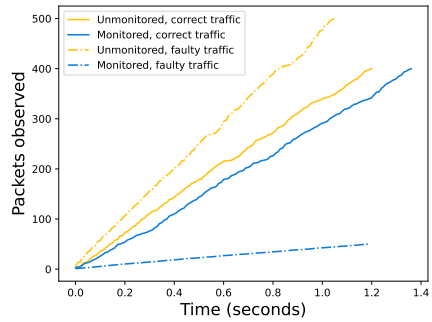
```
1  def protocol(s):
2      isbn = s.recvMsg(Client, Request).text
3      s.sendMsg(Review, RqsReview(isbn))
4      s.sendMsg(Review, RqsDetail(isbn)) # BAD
5      s.sendMsg(Details, RqsDetail(isbn))
6      s.sendMsg(Details, RqsReview(isbn)) # BAD
7      review = s.recvMsg(Review, RspReview).text
8      details = s.recvMsg(Details, RspDetail).text
9      s.sendMsg(Client, Response(review + details))
```

Figure 13 Faulty implementation of the session type in Fig. 5.

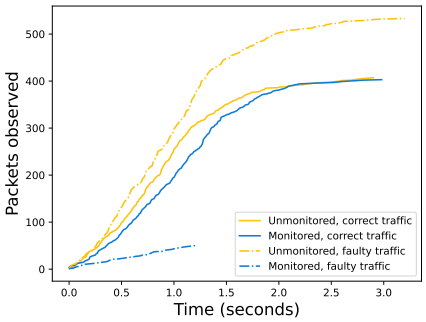
- By contrast, there is no increase in packets observed under monitoring (blue hatched columns), because each faulty packet (which is UDP in this case) is dropped by the session monitor for `Info` and does not enter the network; hence, the faulty packet is not observed by other end hosts.
- With TCP transport and TCP-oriented monitors, incorrect packets cause a *significant* drop in the observed packet counts: this is due to the session-closing mechanism in §3.3, which blocks a session as soon as a faulty packet is observed. In Fig. 14 it is possible to see that the `NEST` TCP monitors keep the observed packet count persistently low in faulty runs, as no further packets are observed after a faulty one causes its session to be closed.
- On unreliable networks with TCP, there are slight differences for observed packet counts between monitored and unmonitored examples. This is because (1) monitors may drop out-of-order packets, which may slightly reduce packet count, or slightly increase it due to retransmissions; and (2) we perform random packet drops, reordering, and delays to simulate unreliable networks. Consequently, some out-of-order and retransmitted packets may be dropped or received by the end host multiple times, causing variations in packet counts. Crucially, the plots show that our TCP monitors do not block correct traffic – otherwise they would disrupt TCP connections and *significantly* drop the "monitored, correct traffic" packet count like the "monitored, faulty traffic" TCP count.

The same trend appears for VPN in the bottom row of Fig. 14. There, faulty packets are sent later than in BookInfo, so monitored faulty runs observe more packets before session closure. Corresponding plots for the remaining examples are available in [46, §C].

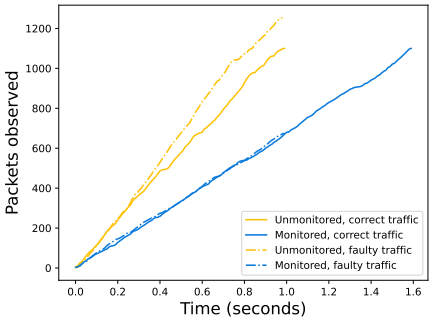
These observations show that our monitors do not disrupt well-behaved programs and correctly reject non-conformant messages, for non-trivial cases: therefore, questions Q 1 and Q 2 can be answered positively. Question Q 3 can be answered positively as well, as these observations hold when running multiple concurrent sessions for each test case.



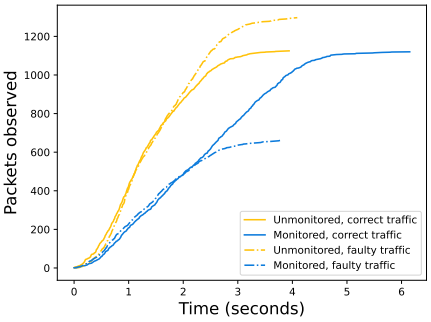
(a) BookInfo, reliable network with TCP.



(b) BookInfo, unreliable network with TCP.



(c) VPN, reliable network with TCP.



(d) VPN, unreliable network with TCP.

Figure 14 Packets observed across all hosts over time for BookInfo (top) and VPN (bottom).

5.3 Monitoring Statistics

Table 2 summarises various packet statistics across the test cases in Table 1:

- For correct implementations (first, third, and fifth columns), monitors reject no packets.
- For faulty implementations (second, fourth, and sixth columns), monitors reject packets.
- With correct programs over TCP on unreliable networks (fifth column), monitors handle retransmissions without introducing wrong rejections.

The statistics support the effectiveness of NEST against questions Q 2 (correct monitoring) and Q 3 (monitoring of parallel sessions). Table 2 also shows that packet retransmissions can happen under TCP even if the protocols are correctly implemented – both on reliable and unreliable networks. Packets may be retransmitted when they are lost (in unreliable networks), or depending on their delivery speed. E.g., the sender's TCP stack may retransmit a packet if an ACK does not arrive quickly enough, or the recipient's TCP buffer may become full and drop some packets, causing their retransmission. NEST correctly handles these situations.

6 Related Work

Session types have been extensively developed in standalone programming languages [26, 27, 19, 68, 36], and as libraries or tools for existing languages such as Rust [37, 41, 15, 16, 12], Java [28, 29], Scala [62], OCaml [57, 31], Haskell [59, 32, 48, 56], Go [11, 55], and others [69].

Table 2 Statistics collected by our `NEST`-generated session monitors for the test cases in Table 1. For each test case, we consider both correct and faulty implementations, using either UDP or TCP as the transport protocol, with TCP evaluated under both reliable and unreliable network conditions. Column **A** reports the total number of packets **A**ccepted by the monitor, **R** the total number of **R**ejected packets, and **T** the number of packets identified as re**T**ransmissions.

	UDP Correct			UDP Faulty		
	A	R	T	A	R	T
VPN	1950	0	0	1950	150	0
Book	800	0	0	800	100	0
Store	1800	0	0	1800	100	0
Firewall	2100	0	0	2100	500	0
DNS	800	0	0	800	100	0
Auction	2800	0	0	2800	200	0
CDN	500	0	0	500	100	0
SIP	350	0	0	350	150	0
POP3	1000	0	0	1000	500	0
Game	3000	0	0	3000	250	0

	TCP + reliable net Correct			TCP + reliable net Faulty		
	A	R	T	A	R	T
VPN	1950	0	0	1125	175	0
Book	800	0	0	100	935	0
Store	1800	0	0	400	450	0
Firewall	2100	0	0	50	531	0
DNS	800	0	0	400	614	0
Auction	2800	0	10	360	995	0
CDN	500	0	0	200	850	0
SIP	350	0	0	200	1102	0
POP3	1000	0	0	500	500	0
Game	3000	0	0	800	447	0

	TCP + unreliable net Correct			TCP + unreliable net Faulty		
	A	R	T	A	R	T
VPN	1950	166	28	1059	510	7
Book	800	3	20	100	712	0
Store	1800	84	40	400	393	4
Firewall	2100	0	21	50	494	1
DNS	800	0	14	400	568	2
Auction	2800	315	52	416	891	2
CDN	500	0	6	200	745	7
SIP	350	48	7	221	827	3
POP3	1000	0	11	500	450	6
Game	3000	106	36	800	509	16

Techniques for enforcing session types with runtime monitors have also been studied in prior work, e.g. [8, 9, 6, 53, 17, 54]. These approaches focus on *application-layer* monitors abstracted from the underlying network. Burlò et al. [8, 9] study binary session types where at least one party is a closed-box process (i.e., not statically verified). They synthesise Scala monitors, prove correctness guarantees, and establish the impossibility of sound and complete black-box monitoring. Bocchi et al. [6] developed a monitored-network framework based on π-calculus processes and multiparty session types. Their "networks" are at a different layer from ours: they model a global routing application (akin to a message broker), implemented with AMQP [67, 17]. That model allows unbounded buffering, so their monitor semantics are close to our session types with queues (Def. 4.3) and do not address the synthesis requirements R 1–R 4 in §4.3 that motivate Def. 4.10.

There is also growing work on runtime enforcement of network properties without session types. For example, `Hydra` [60] deploys "checkers" on P4 switches that enforce network-wide properties. These properties are expressed in terms of packet trajectories through the network and observations of intermediate state at each hop. `FLM` [39] is a language and compiler for enforcing line-rate network monitoring using programmable switches. Our work is complementary: both `Hydra` and `FLM` could serve as implementation platforms for the runtime monitors we propose. At the microservice level, Grewal, Godfrey, and Hsu use runtime monitors to enforce policies [23]. Their goals are similar, but the technical setting differs: they rely on Istio Envoy proxies [33] on end hosts, whereas we target lower-level P4 devices. They also use declarative tree policies, while we use multiparty session types; studying whether a tree-policy-like formalism could model and monitor session protocols is an interesting direction for future work.

Giallorenzo et al. [22] propose choreographic programming projecting (i.e., compiling) executable Java programs that coordinate to perform Virtual Network Functions (VNF) such as intrusion detection and traffic filtering. They present a case study where a P4-enabled virtual switch (based on `BMv2` [13], also adopted in our evaluation) directs network traffic

to the projected VNFs for monitoring purposes. The work [22] is orthogonal to ours: they introduce a high-level software-defined network programming architecture and do not address the problem of tracking session protocols; moreover, their VNFs can implement and run arbitrary code without the constraints of P4-enabled devices (which are a major factor in our work). In principle, the P4 monitors generated by `NEST` could be deployed in the architecture of [22] to perform session monitoring – and their VNFs could deploy and control `NEST` monitors via P4Runtime [14]. A question that links our work to theirs is: is it possible to synthesise `NEST`-style match-action tables from a choreographic program that describes a network-level monitoring policy? This would allow moving the monitoring and filtering logic from their (Java-based) VNFs to P4 devices. This is intriguing and non-trivial work that would require bridging the wide expressiveness gap between choreographic programming languages and P4.

7 Conclusion and Future Work

Conclusion. In this work we addressed the challenge of enforcing session types directly in the network. We developed a formal model of session-type-based monitors, synthesised network-level monitors, and proved correctness under suitable network assumptions. We then designed and implemented `NEST`, which generates (1) session-type monitors for P4-enabled switches and (2) APIs for writing communicating programs tracked by these monitors. Across diverse settings and protocols, our evaluation shows accurate blocking of incorrect messages while allowing correct ones, with low network overhead.

To our knowledge, this is the first work to leverage session types to implement network-level monitors for application-level properties. `NEST` demonstrates that it is possible to automatically synthesise network-level monitors from application-level protocols and deploy these on programmable network switches. Our results have limitations: e.g., our theory requires the input protocols to be finite-state and half-duplex in order to synthesise monitors that are sound yet finite-state – which in turn is necessary for their P4 representation. Still, we demonstrate that even with these restrictions, network-level session monitoring can support complex multiparty protocols.

Future Work. Although our work is a first step toward network monitoring based on session types, several theoretical and practical challenges remain.

Towards monitoring completeness. A natural next step is the dual of soundness (Theorem 4.18): *completeness*, i.e., rejection of all bad messages. Proving completeness requires a precise characterisation of "bad" messages (see Examples 4.7 and 4.11), and [8, Theorem 21] suggests that sound and complete monitoring may be unattainable in our setting. Instead, we conjecture that our monitors are maximally strict: for any $T \neq \mathbf{end}$, if any accepting transition of $[\![T]\!]$ is turned into `reject`, then there exists a network N that satisfies the hypotheses of Theorem 4.18 but not its thesis.

Formalising TCP-oriented monitors. Our formal model (§4) focuses on the "core logic" of network-level session monitoring in an idealised network with perfect message delivery. This abstraction allows us to highlight the differences between our network-level monitors and previous work on application-level session monitoring [6, 17]; extending our formal model to cover TCP-oriented monitors under message duplication, loss, and reordering is valuable and challenging future work. It would require developing a (partial) formalisation of TCP, which is a significant undertaking worth a separate paper, as evidenced by previous work in this area (e.g., [49, 5]). Therefore, we chose to focus our formalisation on the core monitor logic and empirically validate the TCP-oriented extension outlined in §3.3.

Ensuring properties of NEST's input session types. As mentioned in Footnote 1 and Remark 4.19, the current version of NEST assumes that the local session types given as input are part of a multiparty protocol that is output-live (Def. 4.15) and half-duplex. NEST can be extended to check and guarantee these properties, e.g., via bounded model checking, or by interfacing to existing tools (such as Scribble, νScr, mpstk) to project local session types out of global types. This extension would make NEST more user-friendly without impacting its core functionality (i.e., monitor synthesis) and the contributions of this work.

Encryption. End-to-end encryption below the session header is compatible with NEST. However, the current version assumes headers down to the session header are unencrypted, which may leak information. Supporting encryption of packet and session headers is future work, potentially building on P4 encrypted-protocol techniques [24, 25] and homomorphic encryption [20].

Packet fragmentation. Another direction concerns packet fragmentation. The current version of NEST assumes a one-to-one correspondence between session-type messages and network packets. This is often acceptable in modern IP networks with consistent MTUs, but application-level messages can still span multiple packets. NEST and our session API (§3.4) could be extended to support messages spanning multiple packets, while still avoiding network-level fragmentation, by adding a sequence number or flag to the session header (§3.1) to indicate whether the current message continues in the next packet.

Supporting other protocols and consistency models. Finally, we plan to broaden our support for diverse transport and consistency requirements. Our TCP-oriented monitor implementation should adapt to other reliable ordered transports, such as QUIC [35] and SCTP [40].

References

1 Hitesh Ballani, Yatin Chawathe, Sylvia Ratnasamy, Timothy Roscoe, and Scott Shenker. Off by Default! In *ACM Workshop on Hot Topics in Networks (HotNets)*, 2005. URL: https://conferences.sigcomm.org/hotnets/2005/papers/ballani.pdf.

2 Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification – Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi:10.1007/978-3-319-75632-5_1.

3 Massimo Bartoletti, Alceste Scalas, Emilio Tuosto, and Roberto Zunino. Honesty by typing. *Logical Methods in Computer Science*, 12(4), 2016. doi:10.2168/LMCS-12(4:7)2016.

4 Adam D. Barwell, Alceste Scalas, Nobuko Yoshida, and Fangyi Zhou. Generalised multiparty session types with crash-stop failures. In *International Conference on Concurrency Theory (CONCUR)*, volume 243 of *LIPIcs*, pages 35:1–35:25. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.CONCUR.2022.35.

5 Steve Bishop, Matthew Fairbairn, Hannes Mehnert, Michael Norrish, Tom Ridge, Peter Sewell, Michael Smith, and Keith Wansbrough. Engineering with logic: Rigorous test-oracle specification and validation for TCP/IP and the sockets API. *Journal of the ACM*, 66(1):1:1–1:77, 2019. doi:10.1145/3243650.

6 Laura Bocchi, Tzu-Chun Chen, Romain Demangeon, Kohei Honda, and Nobuko Yoshida. Monitoring networks through multiparty session types. *Theoretical Computer Science*, 669:33–58, 2017. doi:10.1016/j.tcs.2017.02.009.

7 Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: programming protocol-independent packet processors. *ACM SIGCOMM Computer Communications Review (CCR)*, 44(3):87–95, 2014. doi:10.1145/2656877.2656890.

8 Christian Bartolo Burlò, Adrian Francalanza, and Alceste Scalas. On the monitorability of session types, in theory and practice. In *European Conference on Object-Oriented Programming (ECOOP)*, volume 194 of *LIPIcs*, pages 20:1–20:30. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ECOOP.2021.20.

9 Christian Bartolo Burlò, Adrian Francalanza, Alceste Scalas, Catia Trubiani, and Emilio Tuosto. Towards probabilistic session-type monitoring. In *Coordination Models and Languages (COORDINATION)*, volume 12717 of *Lecture Notes in Computer Science*, pages 106–120. Springer, 2021. doi:10.1007/978-3-030-78142-2_7.

10 Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *International Conference on Concurrency Theory (CONCUR)*, volume 6269 of *Lecture Notes in Computer Science*, pages 222–236. Springer, 2010. doi:10.1007/978-3-642-15375-4_16.

11 David Castro-Perez, Raymond Hu, Sung-Shik Jongmans, Nicholas Ng, and Nobuko Yoshida. Distributed programming using role-parametric session types in Go: Statically-typed endpoint APIs for dynamically-instantiated communication structures. *Proceedings of the ACM on Programming Languages (PACMPL)*, 3(POPL):29:1–29:30, 2019. doi:10.1145/3290342.

12 Ruofei Chen, Stephanie Balzer, and Bernardo Toninho. Ferrite: A judgmental embedding of session types in Rust. In *European Conference on Object-Oriented Programming (ECOOP)*, volume 222 of *LIPIcs*, pages 22:1–22:28. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ECOOP.2022.22.

13 P4 Language Consortium. BMv2: P4 reference software switch. Available at https://github.com/p4lang/behavioral-model, 2017.

14 P4 Language Consortium. P4runtime v1.5.0 specification. Available at https://p4lang.github.io/p4runtime/spec/v1.5.0/P4Runtime-Spec.html, 2026.

15 Zak Cutner and Nobuko Yoshida. Safe session-based asynchronous coordination in Rust. In *Coordination Models and Languages (COORDINATION)*, volume 12717 of *Lecture Notes in Computer Science*, pages 80–89. Springer, 2021. doi:10.1007/978-3-030-78142-2_5.

16 Zak Cutner, Nobuko Yoshida, and Martin Vassor. Deadlock-free asynchronous message reordering in Rust with multiparty session types. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 246–261, 2022. doi:10.1145/3503221.3508404.

17 Romain Demangeon, Kohei Honda, Raymond Hu, Rumyana Neykova, and Nobuko Yoshida. Practical interruptible conversations: Distributed dynamic verification with multiparty session types and Python. *Formal Methods in Systems Design*, 46(3):197–225, 2015. doi:10.1007/s10703-014-0218-8.

18 Romain Demangeon and Nobuko Yoshida. On the expressiveness of multiparty sessions. In *Conference on Foundation of Software Technology and Theoretical Computer Science (FSTTCS)*, volume 45 of *LIPIcs*, pages 560–574. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015. doi:10.4230/LIPIcs.FSTTCS.2015.560.

19 Simon J. Gay and Vasco Thudichum Vasconcelos. Linear type theory for asynchronous session types. *Journal of Functional Programming*, 20(1):19–50, 2010. doi:10.1017/S0956796809990268.

20 Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, USA, 2009. URL: https://searchworks.stanford.edu/view/8493082.

21 Silvia Ghilezan, Jovanka Pantovic, Ivan Prokic, Alceste Scalas, and Nobuko Yoshida. Precise subtyping for asynchronous multiparty sessions. *ACM Transactions on Computational Logic*, 24(2):14:1–14:73, 2023. doi:10.1145/3568422.

22 Saverio Giallorenzo, Jacopo Mauro, Andrea Melis, Fabrizio Montesi, Marco Peressotti, and Marco Prandini. Choreography-defined networks: A case study on DoS mitigation. In Walid Gaaloul, Michael Sheng, Qi Yu, and Sami Yangui, editors, *International Conference on Service-Oriented Computing (ICSOC)*, volume 15405 of *Lecture Notes in Computer Science*, pages 243–259. Springer, 2024. doi:10.1007/978-981-96-0808-9_18.

23 Karuna Grewal, Philip Brighten Godfrey, and Justin Hsu. Expressive policies for microservice networks. In *ACM Workshop on Hot Topics in Networks (HotNets)*, pages 280–286, 2023. doi:10.1145/3626111.3628181.

24 Frederik Hauser, Marco Häberle, Mark Schmidt, and Michael Menth. P4-IPsec: Site-to-site and host-to-site VPN with IPsec in P4-based SDN. *IEEE Access*, 8:139567–139586, 2020. doi:10.1109/ACCESS.2020.3012738.

25 Frederik Hauser, Mark Schmidt, Marco Häberle, and Michael Menth. P4-MACsec: Dynamic topology monitoring and data layer protection with MACsec in P4-based SDN. *IEEE Access*, 8:58845–58858, 2020. doi:10.1109/ACCESS.2020.2982859.

26 Kohei Honda. Types for dyadic interaction. In *International Conference on Concurrency Theory (CONCUR)*, volume 715 of *Lecture Notes in Computer Science*, pages 509–523. Springer, 1993. doi:10.1007/3-540-57208-2_35.

27 Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 273–284. ACM, 2008. doi:10.1145/1328438.1328472.

28 Raymond Hu, Dimitrios Kouzapas, Olivier Pernet, Nobuko Yoshida, and Kohei Honda. Type-safe eventful sessions in Java. In *European Conference on Object-Oriented Programming (ECOOP)*, volume 6183 of *Lecture Notes in Computer Science*, pages 329–353. Springer, 2010. doi:10.1007/978-3-642-14107-2_16.

29 Raymond Hu, Nobuko Yoshida, and Kohei Honda. Session-based distributed programming in Java. In *European Conference on Object-Oriented Programming (ECOOP)*, volume 5142 of *Lecture Notes in Computer Science*, pages 516–541. Springer, 2008. doi:10.1007/978-3-540-70592-5_22.

30 Stephen Ibanez, Gordon J. Brebner, Nick McKeown, and Noa Zilberman. The P4–>NetFPGA workflow for line-rate packet processing. In *ACM SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 1–9, 2019. doi:10.1145/3289602.3293924.

31 Keigo Imai, Nobuko Yoshida, and Shoji Yuen. Session-ocaml: A session-based library with polarities and lenses. *Science of Computer Programming*, 172:135–159, 2019. doi:10.1016/j.scico.2018.08.005.

32 Keigo Imai, Shoji Yuen, and Kiyoshi Agusa. Session type inference in Haskell. In *Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software (PLACES)*, volume 69 of *Electronic Proceedings in Theoretical Computer Science*, pages 74–91. Open Publishing Association, 2010. doi:10.4204/EPTCS.69.6.

33 Istio. Architecture. Available at https://istio.io/latest/docs/ops/deployment/architecture/, 2025.

34 Istio. Bookinfo Application. Available at https://istio.io/latest/docs/examples/bookinfo/, 2025.

35 Jana Iyengar and Martin Thomson. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000, May 2021. doi:10.17487/RFC9000.

36 Jules Jacobs, Stephanie Balzer, and Robbert Krebbers. Multiparty GV: functional multiparty session types with certified deadlock freedom. *Proceedings of the ACM on Programming Languages (PACMPL)*, 6(ICFP):466–495, 2022. doi:10.1145/3547638.

37 Thomas Bracht Laumann Jespersen, Philip Munksgaard, and Ken Friis Larsen. Session types for Rust. In *ACM SIGPLAN Workshop on Generic Programming (WGP)*, pages 13–22, 2015. doi:10.1145/2808098.2808100.

38 Xin Jin, Xiaozhou Li, Haoyu Zhang, Robert Soulé, Jeongkeun Lee, Nate Foster, Changhoon Kim, and Ion Stoica. NetCache: Balancing key-value stores with fast in-network caching. In *ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 121–136, 2017. doi:10.1145/3132747.3132764.

39 Andrew Johnson, Ryan Beckett, Xiaoqi Chen, Ratul Mahajan, and David Walker. Sequence abstractions for flexible, line-rate network monitoring. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 1593–1620, 2024. URL: https://www.usenix.org/conference/nsdi24/presentation/johnson.

40 Malleswar Kalla, Randall R. Stewart, Tom Taylor, Dr. Vern Paxson, Chip Sharp, Ken Morneault, Dr. HannsJuergen Schwarzbauer, Qiaobing Xie, Ian Rytina, and Lixia Zhang. Stream Control Transmission Protocol. RFC 2960, October 2000. doi:10.17487/RFC2960.

41 Wen Kokke. Rusty variation: Deadlock-free sessions with failure in Rust. In *Interaction and Concurrency Experience (ICE)*, volume 304 of *Electronic Proceedings in Theoretical Computer Science*, pages 48–60. Open Publishing Association, 2019. doi:10.4204/EPTCS.304.4.

42 James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Pearson, 2021.

43 Julien Lange and Nobuko Yoshida. Verifying asynchronous interactions via communicating session automata. In *International Conference on Computer Aided Verification (CAV)*, volume 11561 of *Lecture Notes in Computer Science*, pages 97–117. Springer, 2019. doi:10.1007/978-3-030-25540-4_6.

44 Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In Geoffrey G. Xie, Robert Beverly, Robert Morris, and Bruce Davie, editors, *ACM Workshop on Hot Topics in Networks (HotNets)*, page 19, 2010. doi:10.1145/1868447.1868466.

45 Jens Kanstrup Larsen, Roberto Guanciale, Philipp Haller, and Alceste Scalas. P4R-type: A verified API for P4 control plane programs. *Proceedings of the ACM on Programming Languages (PACMPL)*, 7(OOPSLA2):1935–1963, 2023. doi:10.1145/3622866.

46 Jens Kanstrup Larsen, Alceste Scalas, Guy Amir, Jules Jacobs, Jana Wagemaker, and Nate Foster. NEST: Network Enforced Session Types (Technical Report), 2026. doi:10.48550/arXiv.2604.21795.

47 Elaine Li, Felix Stutz, Thomas Wies, and Damien Zufferey. Complete multiparty session type projection with automata. In *International Conference on Computer Aided Verification (CAV)*, volume 13966, pages 350–373. Springer, 2023. doi:10.1007/978-3-031-37709-9_17.

48 Sam Lindley and J. Garrett Morris. Embedding session types in Haskell. In *International Symposium on Haskell*, pages 133–145, 2016. doi:10.1145/2976002.2976018.

49 Lars Lockefeer, David M. Williams, and Wan J. Fokkink. Formal specification and verification of TCP extended with the window scale option. *Science of Computer Programming*, 118:3–23, 2016. doi:10.1016/j.scico.2015.08.005.

50 Robin Milner. *Communication and Concurrency*. International Series in Computer Science. Prentice Hall, 1989.

51 Mininet. Mininet: Rapid Prototyping for Software Defined Networks. Available at https://mininet.org/, 2022.

52 John G. Myers. POP3 AUTHentication command. RFC 1734, December 1994. doi:10.17487/RFC1734.

53 Rumyana Neykova, Laura Bocchi, and Nobuko Yoshida. Timed runtime monitoring for multiparty conversations. *Formal Aspects of Computing*, 29(5):877–910, 2017. doi:10.1007/s00165-017-0420-8.

54 Rumyana Neykova, Nobuko Yoshida, and Raymond Hu. SPY: local verification of global protocols. In *International Conference on Runtime Verification (RV)*, volume 8174 of *Lecture Notes in Computer Science*, pages 358–363. Springer, 2013. doi:10.1007/978-3-642-40787-1_25.

55 Nicholas Ng and Nobuko Yoshida. Static deadlock detection for concurrent Go by global session graph synthesis. In *International Conference on Compiler Construction (CC)*, pages 174–184, 2016. doi:10.1145/2892208.2892232.

56 Dominic A. Orchard and Nobuko Yoshida. Effects as sessions, sessions as effects. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 568–581, 2016. doi:10.1145/2837614.2837634.

57 Luca Padovani. A simple library implementation of binary sessions. *Journal of Functional Programming*, 27:e4, 2017. doi:10.1017/S0956796816000289.

58 Ivan Prokić, Simona Prokić, Silvia Ghilezan, Alceste Scalas, and Nobuko Yoshida. On asynchronous multiparty session types for federated learning. In *International Conference on Theoretical Aspects of Computing (ICTAC)*, pages 164–182, 2025. doi:10.1007/978-3-032-11176-0_11.

59 Riccardo Pucella and Jesse A. Tov. Haskell session types with (almost) no class. In *ACM SIGPLAN Symposium on Haskell*, pages 25–36, 2008. doi:10.1145/1411286.1411290.

60 Sundararajan Renganathan, Benny Rubin, Hyojoon Kim, Pier Luigi Ventre, Carmelo Cascone, Daniele Moro, Charles Chan, Nick McKeown, and Nate Foster. Hydra: Effective runtime network verification. In *ACM SIGCOMM Conference (SIGCOMM)*, pages 182–194, 2023. doi:10.1145/3603269.3604856.

61 Marshall T. Rose and John G. Myers. Post Office Protocol – Version 3. RFC 1939, May 1996. doi:10.17487/RFC1939.

62 Alceste Scalas and Nobuko Yoshida. Lightweight session programming in Scala. In *European Conference on Object-Oriented Programming (ECOOP)*, volume 56 of *LIPIcs*, pages 21:1–21:28. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPIcs.ECOOP.2016.21.

63 Alceste Scalas and Nobuko Yoshida. Less is more: multiparty session types revisited. *Proceedings of the ACM on Programming Languages (PACMPL)*, 3(POPL):30:1–30:29, 2019. doi:10.1145/3290343.

64 Eve Schooler, Jonathan Rosenberg, Henning Schulzrinne, Alan Johnston, Gonzalo Camarillo, Jon Peterson, Robert Sparks, and Mark J. Handley. SIP: Session Initiation Protocol. RFC 3261, July 2002. doi:10.17487/RFC3261.

65 Justine Sherry, Shaddi Hasan, Colin Scott, Arvind Krishnamurthy, Sylvia Ratnasamy, and Vyas Sekar. Making middleboxes someone else's problem: Network processing as a cloud service. In Lars Eggert, Jörg Ott, Venkata N. Padmanabhan, and George Varghese, editors, *ACM SIGCOMM Conference (SIGCOMM)*, pages 13–24, 2012. doi:10.1145/2342356.2342359.

66 Felix Stutz and Damien Zufferey. Comparing channel restrictions of communicating state machines, high-level message sequence charts, and multiparty session types. In Pierre Ganty and Dario Della Monica, editors, *International Symposium on Games, Automata, Logics and Formal Verification (GandALF)*, volume 370 of *Electronic Proceedings in Theoretical Computer Science*, pages 194–212. Open Publishing Association, 2022. doi:10.4204/EPTCS.370.13.

67 Steve Vinoski. Advanced message queuing protocol. *IEEE Internet Computing*, 10(6):87–89, 2006. doi:10.1109/MIC.2006.116.

68 Philip Wadler. Propositions as sessions. In *International Conference on Functional Programming (ICFP)*, pages 273–286, 2012. doi:10.1145/2364527.2364568.

69 Nobuko Yoshida. Programming language implementations with multiparty session types. In *Active Object Languages: Current Research Trends*, volume 14360 of *Lecture Notes in Computer Science*, pages 147–165. Springer, 2024. doi:10.1007/978-3-031-51060-1_6.

70 Nobuko Yoshida and Ping Hou. Less is more revisited: Association with global multiparty session types. In *The Practice of Formal Methods: Essays in Honour of Cliff Jones, Part II*, volume 14781 of *Lecture Notes in Computer Science*, pages 268–291. Springer, 2024. doi:10.1007/978-3-031-66673-5_14.